

**A Comparison of Multi-Class Classification Models for
NetFlow Anomaly Detection**

Diego Andrade

BSc (Hons) Ethical Hacking, 2026

Word Count: 17,081

School of Design and Informatics

Abertay University

Abstract

In the current enterprise landscape, businesses are becoming more interconnected than ever. This demand is driven by increasing remote-work practices and a fast-evolving technology climate. In large network environments, performing granular packet-level inspection can be impractical. NetFlow aggregates network traffic metadata to monitor the rate of flow, volume, and many other statistics. Importantly, NetFlow data captures these statistics with scoped to pairs of source and destination hosts. This allows for host-level granularity while maintaining a concise high-level representation of traffic. This makes NetFlow the ideal data source for training a machine learning (ML) based Network Intrusion Detection System (NIDS). This report aims to determine whether ML-based NetFlow anomaly detection can be made more actionable and comprehensive by implementing multi-class model outputs and enhancing input features with network-specific IP embeddings.

The artefact yielded by this report has two main parts: an IP embedding neural network (NN), and an anomaly detection framework implementing various multi-class classifiers. Both of these parts will be implemented with the Rust programming language, utilizing a variety of third-party libraries. The final IP embedding NN architecture consists of 34 input neurons projected to 164 neurons, with a 256-neuron fusion layer, and an output layer of size 128. The anomaly detection framework consists of a complex, memory-efficient data structure to hold all the samples and embeddings, a full data preprocessing pipeline, and metric classification through a confusion matrix object.

The embedding model proved to be successful in representing abstract concepts such as similar protocols, common ports and protocol pairs, and subnets; however, the experiments failed to reach a clear optimum configuration. The classifier models significantly outperformed binary classifiers from previous work but lacked performance in classifying malicious classes due to extreme class imbalance in the dataset. Ultimately, it was recognized that the Rust ML ecosystem is still to immature for a project of this scale, despite the artefact being completed.

Table of Contents

Table of Figures	iv
Table of Tables.....	v
Acknowledgements	vi
1 Introduction.....	1
1.1 Aims.....	3
1.2 Scope.....	4
1.3 Overview of Remaining Chapters.....	4
2 Literature Review	5
3 Methodology.....	8
3.1 Technical Resources	8
3.1.1 Hardware Specification	8
3.1.2 Programming Language	9
3.1.3 Embedding Model Dependencies	10
3.1.4 Classifier Dependencies.....	12
3.2 Preparation Work	12
3.3 Design Structure	15
3.3.1 Dataset	15
3.3.2 Embedding Model	19
3.3.3 Classification Models	25
3.4 Development.....	28
3.4.1 Embedding Model	28
3.4.2 Classification Models	35
3.5 Experiment Procedure	45
3.5.1 Embedding Model	45
3.5.2 Classification Models	47
3.6 Statistical Analysis	49
3.6.1 Embedding Model	49
3.6.2 Classification Models	49

4	Results.....	51
4.1	Preparation Work	51
4.2	Architectural Hyperparameters	52
4.3	Embedding Hyperparameter Sweep	55
4.4	Classification Models	55
5	Discussion	59
5.1	Architectural Hyperparameters	59
5.2	Embedding Hyperparameter Sweep	60
5.3	Classification Models	61
6	Conclusion	64
6.1	Future Work.....	65
	List of References	67
	Appendices	1
	Appendix A – Batching Data Flow Diagram.....	1
	Appendix B – Embedding Hyperparameter Sweep	2
	Appendix C – Full Metrics per Model.....	3

Table of Figures

Figure 1.1-1: Revenue of Enterprise Network Infrastructure Market Worldwide (Statista, 2026)	1
Figure 1.1-2: Main Network Security Technologies Currently in Use or Planned for Acquisition Worldwide (CyberEdge Group, 2025)	2
Figure 3.3-1: Benign vs. Malicious Distribution by Dataset	18
Figure 3.3-2: Initial Model Architecture	20
Figure 3.3-3: Corrected Model Architecture	21
Figure 3.3-4: Final Model Architecture	22
Figure 3.3-5: MNIST Autoencoding Results (Hendrycks and Gimpel, 2023)	23
Figure 3.4-1: Sample Design Architecture	29
Figure 3.4-2: Generation of training samples (Ring et al., 2017).....	30
Figure 4.1-1: Linfa Logistic Regression Output.....	51
Figure 4.1-2: SmartCore Logistic Regression Output.....	51
Figure 4.2-1: Initial PCA-2 Projections (All)	52
Figure 4.2-2: Initial PCA-3 Projection (Protocol).....	53
Figure 4.2-3: Initial PCA-2 Projection, Single Subnet (Port and Protocol).....	53
Figure 4.2-4: Corrected PCA-2 Projection (All)	54
Figure 4.2-5: Corrected t-SNE-3 Projection, Coloured by Subnet (/24)	54
Figure 4.4-1: Linear SVM Metrics by Class.....	56
Figure 4.4-2: GBDT Metrics by Class	56
Figure 4.4-3: Multi Cascading GBDT Metrics by Class	57
Figure 4.4-4: One vs. Rest Cascading GBDT Metrics by Class	58

Table of Tables

Table 3.2-1: Experimental Variables for Preliminary Experiment	13
Table 3.5-1: Experimental Variables for Training Hyperparameter Sweep	46
Table 3.5-2: Possible Values of Independent Variables for Training Hyperparameter Sweep.....	46
Table 3.5-3: Experimental Variables for Classifier Evaluation	48
Table 4.1-1: Preliminary Experiment Performance Results	52

Acknowledgements

Quisiera agradecer a toda mi familia, y especialmente a mis padres, por todo lo que han sacrificado por mi y mi hermano toda nuestra vida. Gracias por todo su cariño y soporte, me han ayudado mas de lo que saben para llegar aquí.

También quiero agradecer a mi abuelo Gonzalo, por enseñarle a mi mamá que la mayor herencia para los hijos es la educación.

Thank you Hebe, for your support, I could not have done this without you.

1 Introduction

In the current enterprise landscape, businesses are becoming more interconnected than ever. This demand is driven by increasing remote-work practices and a fast-evolving technology climate. In a study published by Microsoft about network traffic dynamics in their own Microsoft Research Cambridge enterprise network, they collected over 13 billion unique network packets worth 12.5TB of data in just three and a half weeks (Karagiannis and Mortier, 2008). In a network environment like this, volume is not the only issue; networks produce large amounts of data at a continuous and high velocity. Performing granular packet-level inspection at this scale can be impractical, especially for real-time detection. Using signature or rule-based analysis is highly susceptible to packet fragmentation and is hard to maintain. Additionally, since 2008, network infrastructures have become more complex and distributed due to increased cloud adoption.

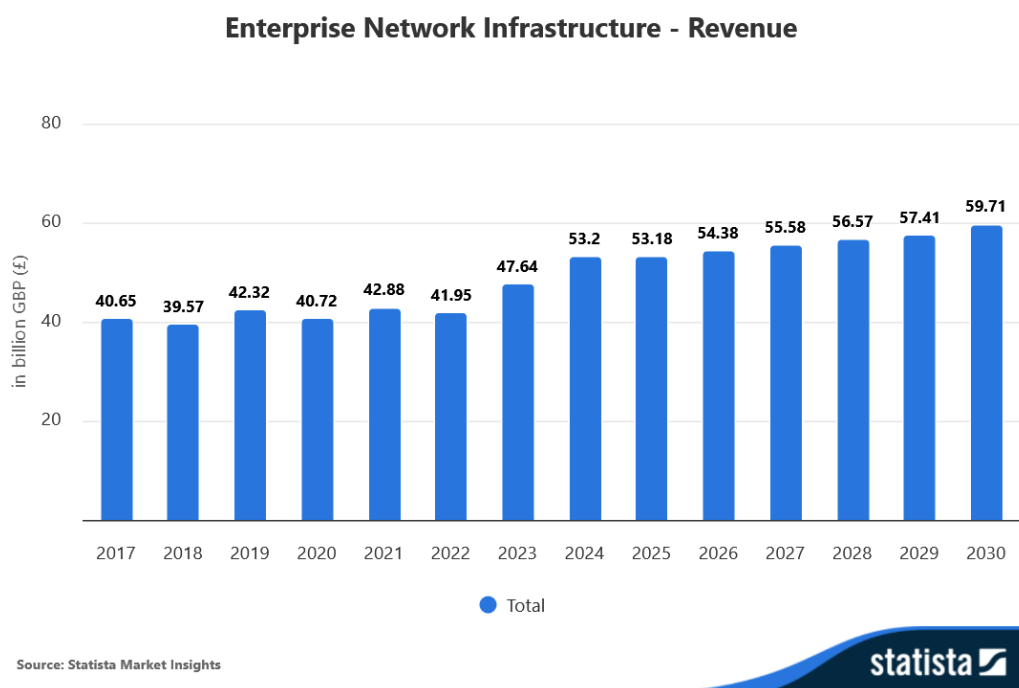


Figure 1.1-1: Revenue of Enterprise Network Infrastructure Market Worldwide (Statista, 2026)

Figure 1-1 shows how spending in the enterprise network infrastructure market has grown and is forecast to demonstrate an annual growth rate of 4% by 2030 (Statista,

2026). These factors have led to widespread adoption of Network Intrusion Detection Systems (NIDS). Figure 1-2 shows the most used network security technologies currently in use or planned for use in 2024.

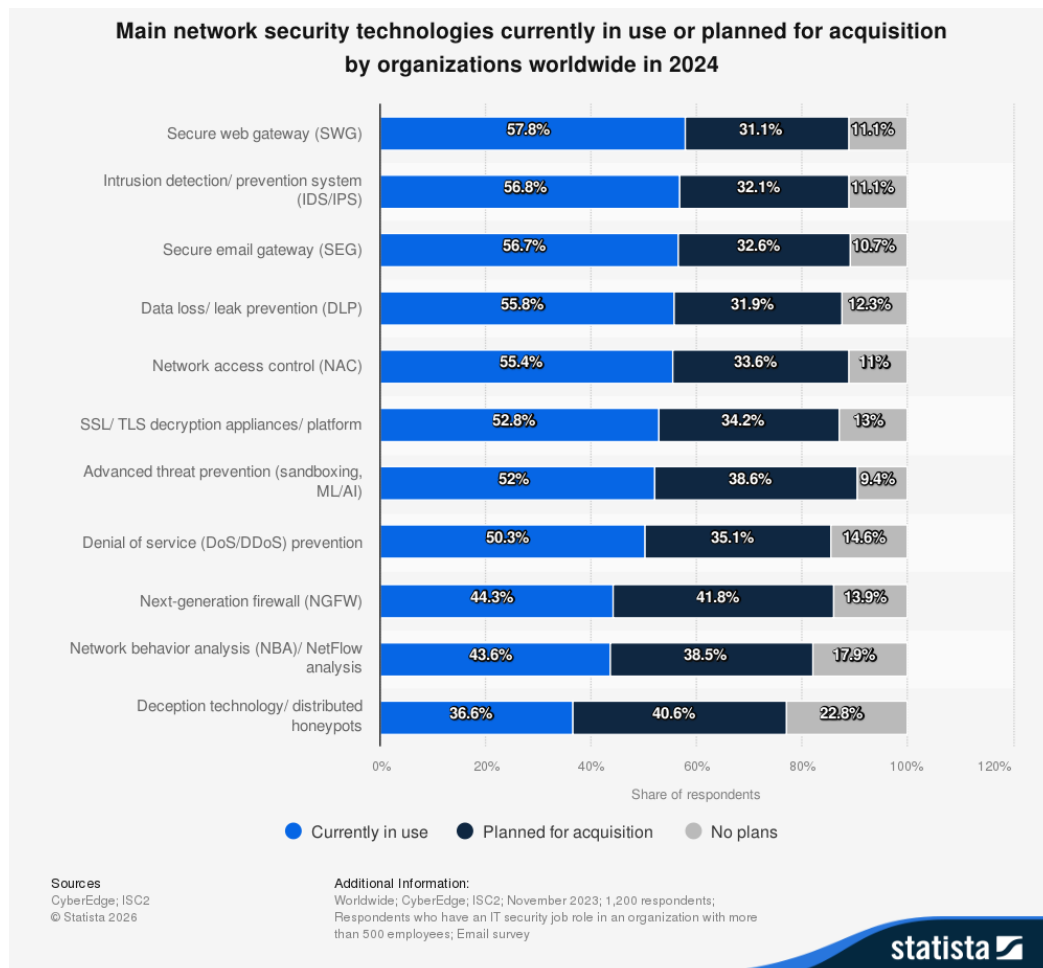


Figure 1.1-2: Main Network Security Technologies Currently in Use or Planned for Acquisition Worldwide (CyberEdge Group, 2025)

As shown above, IDS systems have the second largest share of usage in 2024. Notably, NetFlow analysis is the second least used. This is a key gap, since NetFlow provides an efficient and concise way of quantifying traffic data compared to raw packet captures. NetFlow aggregates network traffic metadata to monitor the rate of flow, volume, and many other statistics. Importantly, NetFlow data is most commonly unidirectional, and captures these statistics scoped to source and destination host pairs. This allows for host-level granularity while maintaining a high-level representation of traffic. It is important to note that in Figure 1-2, NetFlow analysis has a larger percentage of

organizations planning to acquire the technology. Many NIDS provide less than optimal coverage due to outdated techniques such as signature-based and protocol-based analysis. Due to this, machine learning (ML) techniques are emerging as better alternatives. ML allows for adaptive detections and can be more resilient to previously unseen attacks. Furthermore, ML-based NIDS reduce reliance on handcrafted signatures, which are necessary for both signature and protocol-based solutions. The SANS 2024 AI survey collected responses from approximately 470 different organizations and found that 56.9% of respondents answered that they were using AI technology for anomaly detection systems, the highest proportion among all options (Edmonson and Bromiley, 2024). However, the survey also identified various challenges with AI adoption in cybersecurity. 41.8% of respondents said that they had difficulty trusting AI decision due to lack of transparency in the decision-making process. Furthermore, Edmonson and Bromiley mention: “approximately 71% reported AI systems generating false positives, leading to alert fatigue” (2024).

1.1 Aims

This dissertation aims to answer the following research question:

Can ML-based NetFlow anomaly detection be more actionable and comprehensive by implementing multi-class model outputs and enhancing input features with network-specific IP embeddings?

To this end, this project will yield an artefact with two main parts: an IP embedding neural network, and an anomaly detection framework implementing various multi-class classifiers. This research aspires to close the gaps identified in the available NIDS implementations by utilizing multi-class models for greater transparency and using IP embeddings to bolster performance and reduce false positive rates. Additionally, this report will determine which model performs the best and compare with previous work in ML-based anomaly detection.

1.2 Scope

The implementation detailed in this report will focus on comparing the ML models purely on classification performance by evaluating a range of metrics. This means that integration with Security Information and Event Management (SIEM) solutions, real-time detection, and alerting mechanisms are all out of scope for this project.

1.3 Overview of Remaining Chapters

Section 2 explores and evaluates previous work in the ML-based NetFlow anomaly detection field. This section covers three main topics: unsupervised algorithms, supervised binary classification algorithms, and the use IP data (or the lack thereof) in anomaly detection implementations.

Section 3 describes the implementation of the project artefact. Starting with defining the technical resources used, and preparation work that was done before the full implementation. Then, the design, development, experimentation, and evaluation of the artefact are described in detail. These sections are generally split into subsections for the embedding model and the classification models.

Section 4 presents the results of the preparation work, the two stages of the embedding model analysis, and the classification model comparisons.

Section 5 interprets the results presented in section 4 and discusses the implications of the findings. This section focuses on identifying trends, evaluating performance, and determining rationale for observed outcomes.

Section 6 draws conclusions from the elements established in section 5, and outlines where this research has succeeded and where it has failed. Finally, this section will lay out potential paths for future work on this topic.

2 Literature Review

This section will discuss various papers relating to network anomaly detection. These papers were split into five topics: unsupervised vs supervised learning techniques, binary classification models, multi-class classification models, IP data as input features, and deep learning for anomaly detection.

Although behavioural IDS anomaly detection typically requires training data (e.g. supervised learning), unsupervised methods can be effective. One unsupervised approach was proposed by Flanagan *et al.* (2017), where they introduced a novel clustering method that evaluated cluster density over time. This was different to the commonly seen k -means clustering, which calculates static point distance at a fixed point in time. They tested their approach on a live business environment over a 24-hour period; however, they found no real malicious traffic, so there is no evidence that it can detect malicious techniques. They did uncover some misconfigurations and unusual behaviour, despite ultimately being benign or employee traffic. This work is likely faster than supervised solutions discussed later, but the authors did not include any runtime metrics in their paper. This work can be likened to a regression model, which utilizes a statistical baseline (cluster density) to flag anomalous events. Techniques like this are highly prone to false positives, as illustrated by the results of the paper. This research could be improved by testing on a testbed environment with simulated attacks, to prove the algorithm can detect threats on the network. A literature review of ML techniques used in the cybersecurity space found that while clustering has a possibility of detecting previously unseen attack methods, they have a higher false positive rate than supervised algorithms (Roponen *et al.*, 2021). This study also found that in the reviewed papers, clustering had a lower best-performance than supervised algorithms for “traffic flow-based anomaly detection”. Due to these factors, this project elected to use a publicly available training dataset with malicious training samples that resemble real network attacks, ensuring that the models can learn from real attack patterns.

Previous studies of ML-based NetFlow anomaly detection have only dealt with binary classification techniques. This means that the models only classify flows as benign or malicious and leave identification and diagnosing to human reviewers. This makes binary classifiers ill-suited for use in an industry context, because information returned by the models should be quickly actionable. Fosić *et al.* (2023) make a clear decision to convert the UNSW-NB15 dataset's nine attack types into binary labels. This is because their paper is largely focused on dataset imbalance and using different techniques to reduce the dataset as much as possible whilst retaining model performance. This approach failed to address the usability of the technology in an enterprise environment, where actionable insights are more valuable than utilizing as few resources as possible; especially with many companies expanding their computational capability due to the rise of AI in business.

A search of literature revealed few studies which attempted multi-class classification. One of these studies compared both binary and multi-class supervised algorithms, yet the dataset used had only four attack classes and some models completely failed to detect some classes (Gonzalez Granadillo and Kaaniche, 2025). The authors concluded that this was due to a highly imbalanced dataset; however, they managed to improve their worst-class scores by creating an ensemble of all five multi-class models. This suggests that the issue lied in their feature engineering, as different models were better or worse at different classes. This report aims to provide a more structured and effective dataset preprocessing pipeline, which should be able to perform uniformly and provide a consistent foundation across models.

Of particular concern is the lack of IP data in all reviewed studies. These studies state various reasons for exclusion of IP data, such as: DHCP providers are likely to change the IPs of hosts frequently, IPs do not carry over across environments so trained models are not reusable, and some entirely discount the need for IP data by declaring it insignificant (Flanagan *et al.*, 2017; Gonzalez Granadillo and Kaaniche, 2025; Fosić *et al.*, 2023). This dissertation contends that the relationships between network entities are important data and invaluable for anomaly detection in network traffic. To this end, a couple papers

adapted the well-known Word2Vec algorithm (Mikolov *et al.*, 2013), normally used to represent words for ML text interpretation, to embed IP addresses into n -dimensional vectors which are usable in ML models utilizing other IPs as context to be predicted by their neural network (NN) (Ring *et al.*, 2017; Koda *et al.*, 2020). The work done by Ring *et al.* is the first study to utilize neural networks to generate latent space embeddings, rather than utilize the more common graph-node approach. Koda *et al.* utilized these embeddings in conjunction with random Fourier features to train an SVM model to identify anomalous IP addresses in network traffic logs. The main limitation of these implementations is that they overlook what happens when a downstream classifier encounters a previously unseen host, a gap this dissertation aims to address.

Vo *et al.* (2024) and Gonzalez Granadillo and Kaaniche (2025) both attempted to use deep learning models in their studies and observed that they reached similar levels of performance to their best models but took significantly more time to train and infer. Thus, they concluded that NNs did not provide the balance of performance and computational efficiency that they expected. Nevertheless, this report argues that this application of deep learning NNs was misguided. When using a NN for generating IP embeddings, the embedding model should only be trained periodically and less frequently than a full anomaly detecting classifier. This should allow for the classification performance improvements with significantly less training over time.

To summarize, this project will experiment with supervised, multi-class classification algorithms which utilize a separate NN to embed continuous IP data to usable input features. This will allow this dissertation to provide adequate contrast and substantial improvements to previous work.

3 Methodology

This methodology section will be structured in three main parts: technical resources (Section 3.1), preparation work (Section 3.2), and design & development (Section 3.3 – 3.6). The technical resources section will outline the hardware in which this project was developed and tested, the programming language chosen, and the third-party dependencies used. The preparation work section describes a preliminary experiment conducted before artefact implementation and its consequences on the final design of the artefact. The design & development sections contain at least two subsections. This is because this project is split into two main parts. The first is the embedding model, whose purpose is to embed contextual IP data from the raw dataset and embed it into a latent high-dimensional space which is easier for models to generalize. The second is the classification models, which will detail the implementation of the downstream classifiers, which will take the IP embeddings and other dataset features to actually classify network traffic.

3.1 Technical Resources

3.1.1 Hardware Specification

The experiments described in this report were conducted on two separate hosts, a personal development machine and an AI workload server dubbed “Linebreaker”. The development machine was an MSI GS66 Stealth 11UG laptop with the following specifications:

- CPU: Intel i9-11900H @ 2.5GHz (16 threads)
- GPU: NVIDIA GeForce RTX 3070 Laptop
- RAM: 32GB DDR4
- OS: NixOS version 25.11 (64-bit).

Whilst the “Linebreaker” server was a JupyterHub container running on a server with the following specifications:

- CPU: AMD EPYC 9334 @ 2.7 GHz (128 threads)
- GPU: 2x NVIDIA H100 Tensor Core
- RAM: 377GB
- OS: JupyterLab 4.1.5 (64-bit).

3.1.2 Programming Language

The project artefact was developed using the Rust programming language. The author's previous experience with Rust for developing CLI tools played a part in this choice; however, the language provides many advantages to Python and other languages. The most well-known feature of the Rust programming language is its ownership system. The compiler essentially prevents most types of memory mismanagement errors. The compiler automatically frees memory when a variable drops out of scope, which leads to certain rules being enforced. These rules force the programmer to write more efficient, safer code.

Rust also provides powerful data structures and abstractions which were leveraged in this project to balance performance and brevity. For example, HashSets were used to store all the unique dataset features for the embedding model. HashSets are functionally the same as HashMaps utilizing only keys and no values. This ensures that there is no data duplication. Then, each dataset sample is reconstructed from references to elements within these HashSets, to avoid data duplication between samples entirely. Since both the samples and feature sets are stored on the same struct, this leads to a self-referential Struct, which is not allowed in Rust. This is because it breaks one of the cardinal rules of borrowing in Rust, references must *always* be valid. If a Struct contains a reference to its own field, when the Struct is moved, the references become dangling (since they will point to the previous address in memory). This can be remediated with Rust's Arc and Rc abstractions (Atomic Reference Counter and Reference Counter, respectively). These two types are essentially reference counted pointers; this means that ownership of the data can be shared. When an Arc or Rc is cloned, a new pointer is made to the data, and the reference count is incremented. When the cloned Arc or Rc

goes out of scope, the reference count is decremented. This means that the data will remain allocated until all references to it go out of scope (and the reference count reaches zero). See Section 3.3.1 for further details on how this was implemented for the IP2Vec embedding model.

3.1.3 Embedding Model Dependencies

The IP2Vec embedding model required a specialized deep-learning framework, as it was designed to be an Artificial Neural Network (ANN) rather than use a classical regression model. For this, the *burn* library was selected. It is a newer, less mature library than alternatives such as *tch* (Rust wrapper for *PyTorch* C++ API), but it uses more idiomatic Rust concepts such as a builder pattern for configuration. Additionally, *burn* supports interchangeable backends that allow the model to run on a range of different hardware. Initially, the backend chosen for development was the native CUDA backend; however, the administrators of the AI workload server utilized for training (see Section 3.1.1 for hardware specification), decided that the full CUDA toolkit was out-of-scope for the container image. Thankfully, the *burn* backend system allowed the author to switch the backend to run from the *LibTorch* (*PyTorch* C++ API) toolkit. The toolkit was already installed on the server, and end-users could install it as a standalone server. This, of course, comes with some performance overhead, but the author determined it was minimal compared to the performance gained with the server’s powerful GPUs.

For model serialization and deserialization, within the Rust ecosystem, *Serde* is the de-facto standard. *Serde* provides *Serialize* and *Deserialize* traits that can either be automatically implemented through a *derive* macro annotation or manually implemented. When a struct/type implements either of these traits, they can be serialized and deserialized (accordingly) into any format which supports *Serde*. Since *Serde* is a mature and established part of the Rust ecosystem, *Serde* format libraries are available for nearly any format. In this project, the *rpm-serde* (for *MessagePack*) and *CSV* libraries were used to read and write fitting files and dataset files, respectively. *MessagePack* was chosen as the fitting file format due to its small size and rapid

deserialization capabilities. This was informed by a study conducted at the University of Houston comparing the performance of various serialization formats (Casey, 2022). Casey concludes that the MessagePack format is both more memory efficient and faster to serialize and deserialize (2022, p. 70).

For the Command Line Interface (CLI), two libraries were used: *CLAP* (Command Line Argument Parser) and *Anyhow*. *CLAP* is another staple of the Rust ecosystem; it provides both a derive macro and a manual builder for parsing arguments from the command line. The derive macro functionality offers enough flexibility to be able to define an argument struct which can include subcommands and can automatically parse most data types (even non-standard library types). Additionally, *CLAP* generates a help command which describes each option through parsing of doc comments on each field of the struct. *Anyhow* is an error handling library which homogenizes Result objects to use the same Err type. This allows the various error types from various libraries to propagate to the command line gracefully and with various configurable layers of context. To that end, *Anyhow* allows the developer to add custom context to errors for better user experience. Using an error handling library avoids panicking, which often does not provide adequate context (since it is an abrupt crash) and can often provide unnecessary and sometimes inaccurate information.

For the dimensionality reduction techniques described in Section 3.3.2, two extra libraries were used, since *Burn* does not include this functionality. For fast PCA reduction, the *Petal-Decomposition* library was used. Specifically, the RandomizedPca struct was used to compute the PCA projection, which significantly reduces runtime, at the cost of slight accuracy loss. This was determined to be an acceptable compromise for this use case. For t-SNE reduction, the *BHTSNE* library was used. Again, this library implements the Barnes-Hut variation of t-SNE, which reduces runtime at the cost of slight accuracy, this can be configured through the θ parameter. Furthermore, the *BHTSNE* library implements CPU parallelisation with *rayon* to speed up computation.

Finally, for testing, the *Proptest library* allowed the developer to write tests which are automatically fuzzed based on either default or custom strategies. The *Proptest-Derive* library provides the Arbitrary trait, which can be derived on any struct to automatically implement a fuzzing strategy based on the field types. When unit testing, this can greatly help in detecting and fixing edge-cases.

3.1.4 Classifier Dependencies

For the classifier implementation, the same basic dependencies are reused: *anyhow* for error handling, *serde* for deserialization, and *proptest* for testing. The main difference is the use of *ndarray* to handle data structures. *ndarray* provides n -dimensional matrices and general-purpose mathematical functions to manipulate them. In conjunction with *ndarray*, the *ndarray-linalg* library provides linear algebra functions utilized for feature scoring and statistics, and it allows for integration with the *OpenBLAS*, an implementation of Basic Linear Algebra Subprograms (BLAS) for ultra low-level and highly optimized matrix arithmetic. For reasons described in section 3.4.2.2, the *SmartCore* and *Linfa* libraries were used briefly but proved to be unfit for purpose. Therefore, the *LIBLINEAR* C++ library with the Rust *liblinear* bindings was utilized for implementing a linear SVM model, and the *XGBoost* C++ library with the Rust *xgb* bindings was used for implementing the GBDT models.

3.2 Preparation Work

To prepare for the implementation of the artefact yielded by this research, the author compared three libraries implementing the classical classification machine learning algorithms used for this project. This preliminary experiment uses a controlled experimental design to maintain comparative validity between the three libraries. These libraries were as follows: *scikit-learn* for Python, *Linfa*, and *SmartCore* for Rust. *scikit-learn* was picked due to its widespread popularity in the data science and machine learning fields. This library provides a mature ecosystem and an all-in-one approach to machine learning. It contains modules for: classification tasks, regression tasks,

clustering tasks, dimensionality reduction, model metrics, and dataset preprocessing. These modules contain well-implemented models and algorithms with a variety of hyperparameter options for customization, which make it the current leader in the machine learning academic space. *Linfa* and *SmartCore* were the two clear contenders for an all-in-one Rust solution to classical machine learning. Both libraries are regularly updated and are in active development. They offer similar features to *scikit-learn*; nevertheless, they both follow significantly different design philosophies.

A small experiment was repeated once for each library mentioned. The aim for this experiment was to generate a simple dummy dataset with two blobs of random points and train a logistic regression model to linearly classify the dataset. See Table 3.2-1 for a summary of experimental variables.

Variable	Variable Type	Description
Library	Independent	Classical machine learning library used in logistic regression implementation
Execution Time	Dependent	Time required to generate dataset, fit model, and plot points with decision boundary
Memory Usage	Dependent	Peak memory allocated during runtime
API Import Count	Dependent	Number of imported objects
Dependency Count	Dependent	Total number of required packages
Graphing Library	Control	<i>plotters</i> for Rust libraries and <i>matplotlib</i> for Python
Model Algorithm	Control	Logistic regression
Decision Boundary Algorithm	Control	$0 = w_1x_1 + w_2x_2 + b$

Table 3.2-1: Experimental Variables for Preliminary Experiment

The decision boundary algorithm can be explained by considering the linear predictor equation of a logistic regression model. Given z is the output logit of the function, w is the learned weight vector, x is the input feature vector, and b is the learned bias:

$$z = w^T x + b.$$

For a 2-dimensional linear classifier, this can be expanded to:

$$z = w_1 x_1 + w_2 x_2 + b.$$

This equation represents the output of the model, with $z = 1$ representing 100% confidence that a point (x_1, x_2) belongs to class 1 and $z = -1$ representing 100% confidence that the same point belongs to class 0. To find the decision boundary of the model, one must plot a line where $z = 0$.

Based on the results in Section 4.1, the author decided *SmartCore* was the most fit-for-purpose library available as of writing. Both the Rust libraries significantly outperformed *scikit-learn* in execution time and memory usage. As Rust is a more verbose language compared to Python, naturally *scikit-learn* required less lines of code than both Rust libraries. Nonetheless, with higher complexity, the strongly typed data structures used by both Rust libraries reduce the amount of code needed to develop a maintainable and safe structure. Between the two Rust libraries, *Linfa* proved to have an obtuse API which was hard to use and understand, this was largely due to the way it was structured. The solution was separated into 19 different libraries which had to be individually imported. This resulted in the documentation also being separated into 19 different web pages. *SmartCore* succeeded in being more user-friendly, with more ergonomic paradigms that allowed for easier implementation. The functionality was separated into sub-modules rather than individual libraries, maintaining memory efficiency yet removing the awkward import process. The documentation also benefitted from the library structure, as it could all be found in one place.

3.3 Design Structure

3.3.1 Dataset

Choosing an appropriate dataset is crucial in ensuring any model can learn and generalize information properly. A few critical dataset metrics were kept in mind when selecting a dataset. First, multi-class attack labels are a necessity to satisfy the research question's focus on expanding IDS output to describe the type of detected malicious activity. This is important to differentiate this report's research from previous work as discussed in Chapter 2. Second, a large enough dataset size is important to allow models to generalize and prevent underfitting. Dataset size describes both total network flows and a significant amount of malicious traffic. A common problem with IDS/IPS datasets is a high class imbalance. This means that the datasets contain a significantly larger proportion of benign/normal samples than malicious/anomalous samples. Although this problem is hard to avoid when simulating a real enterprise environment, dataset creators should strive to strike a balance between realism and anomalous sample size. Finally, it is critical that the selected dataset includes meaningful IP information in each flow. This means that there should be an underlying network structure and all samples should be derived from traffic of the same network. Thus, any agglomerated datasets were omitted from consideration, since combining multiple datasets with multiple network structures renders IP context meaningless. Without this, there would be no point in utilizing the IP2Vec embedding model to enhance the dataset with contextual IP information. With these criteria in mind, five datasets were considered for the implementation of the project artefact.

DEDALE (Lanvin and Majorczyk, 2025) is a dataset geared towards detecting medium-term APT campaigns in a multi-zone corporate network. This dataset spans 28 days with completely benign traffic for a few days before and after the orchestrated attack. This means the dataset has a large flow size, but having only one multi-stage attack (despite occurring over multiple weeks) limits the anomalous data. Additionally, since this dataset only captures one attack chain, the range of attack types were severely limited.

The main element that precluded the use of this dataset was the binary labels, which makes the multi-class nature of this research impossible without meticulous re-labelling.

The CasinoLimit (Kilian *et al.*, 2025) dataset is a dataset comprising of 114 attackers attempts at completing a CTF challenge. All instances of the dataset (one per attacker) are isolated, and no attacks from any instance affect any other instance. This dataset initially stood out due to its MITRE ATT&CK labels, which provide descriptive and standardized labels to link network flows with specific attack tactics, techniques, and procedures (TTPs). Despite this, the main limitation of this dataset is the lack of benign traffic. There is no simulated user traffic, which makes establishing a baseline of normal traffic difficult. This ultimately leads to poor anomaly separability, which can cause poor model performance.

Another dataset which features MITRE TTP labels is Unraveled (Myneni *et al.*, 2023). Similarly to DEDALE (Lanvin and Majorczyk, 2025) mentioned earlier, this dataset focuses on APT threats. This contains three attacks simulated with three different skill levels: amateur, skilled, and APT. This still limits the attack types to a handful of attack chains. Furthermore, the distribution of malicious to benign attacks is severely skewed. Only around 1.4% of the network flows are labelled as malicious. As discussed above, this can lead to underfitting. This is especially true in certain parts of the defined network structure. One example is the “Marketing” subnet, which only contains 29 attack flows, representing 0.1% of the subnet traffic.

The CSE-CIC-IDS2018 (Sharafaldin, Habibi Lashkari and Ghorbani, 2018) dataset is an evolution of the preceding CIC-IDS2017 (Sharafaldin, Habibi Lashkari and Ghorbani, 2018) dataset. The former contains a significantly larger number of flows than the latter due to the increased number of hosts in the overall network structure. Both these datasets were automatically labelled from raw packet capture files using the bespoke CICFlowMeter tool developed by the Canadian Institute of Cybersecurity (CIC). These datasets were considered the academic standard for NIDS datasets alongside the

UNSW-NB15 dataset (Moustafa and Slay, 2015), discussed below. Both datasets meet all the criteria defined in the first paragraph of this section; however, various reports uncovered issues with the CIC-IDS2017 dataset, implicating the same issues in CSE-CIC-IDS2018. In a critical review of both datasets, Liu *et al.* (2022) found that the CICFlowMeter tool caused 7.53% of flows to be inaccurately labelled and some attack labels to be corrupted by over 75% in both datasets. Furthermore, Liu *et al.* (2022, s. 6) warn:

Alongside inherent limitations of datasets generated in synthetic environments, our work is unable to remediate issues at the raw PCAP level. For example, the misimplementation of the DoS Hulk attack renders it completely ineffective. This requires re-execution with corrected implementation tools on the original testbed architectures.

These findings imply that any adaptations or re-labelling of the original CSE-CIC-IDS2018 should still be avoided due to issues with the source data generation and collection.

The University of New South Wales (UNSW) developed one of the most popular NIDS datasets, UNSW-NB15 (Moustafa and Slay, 2015). This dataset features the IXIA Perfect Storm traffic generator, which simulates both normal and anomalous traffic originating from multiple servers. All malicious traffic is labelled as one of the following nine attack types: fuzzers, analysis, backdoors, DoS, exploits, generic, reconnaissance, shellcode, and worms. Moreover, the dataset also contains a “ground truth” file which defines each attack timeframe and labels each attack with a sub-category, title, and a link to a specific CVE. The dataset manages to create an increased ratio of benign to malicious samples (around 14.5%) by elevating the attack rate to a non-realistic level of around one attack per second and subsequently ten attacks per second. This manages to create a more effective balance of realistic benign behaviour and enough malicious flows for proper training. The paper accompanying this dataset also provides a general network structure which is enough to ensure that IP context is meaningful. The principal limitation with this dataset is the low total sample size, being the smallest of all five aforementioned

datasets. Nevertheless, the high-quality labels and a balanced ratio of benign to malicious samples illustrate why it is a mainstay of ML NIDS research. See Figure 3.3.1-1 for comparison of percentage distribution of benign to malicious flows across all five datasets.

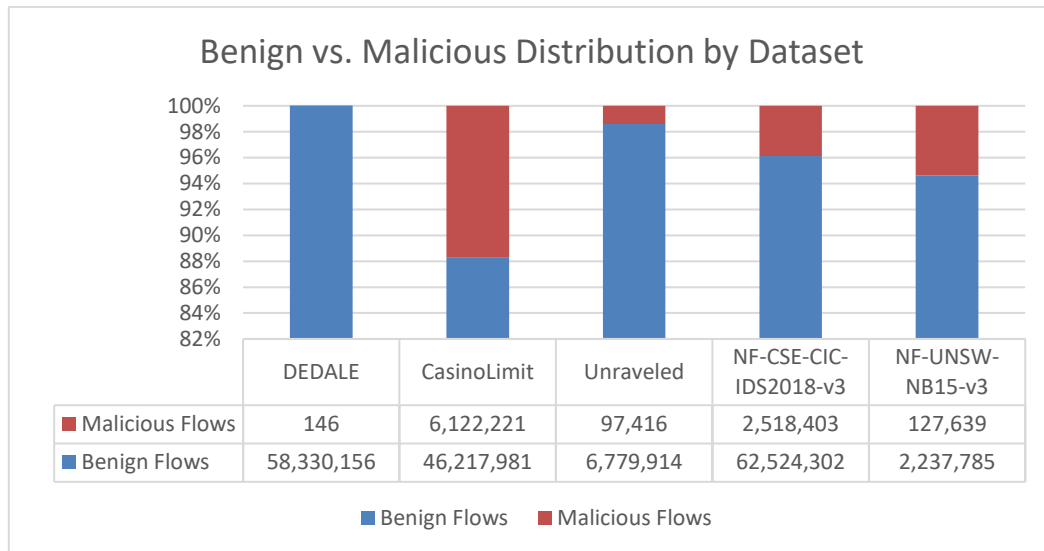


Figure 3.3-1: Benign vs. Malicious Distribution by Dataset

It is important to note that for both the CSE-CIC-IDS2018 and UNSW-NB15 datasets, there exists an enhanced version, which extend the datasets with novel temporal features (Luay *et al.*, 2025). These datasets are prefixed with “NF-” and were developed by the University of Queensland. These were preferred over the standard versions of the datasets as they provide a larger feature space and represent a more modern approach to interpreting raw packet captures into NetFlows. At the time of writing, there are three versions of the updated NetFlow format, marked with a version suffix. The version utilized in this report was version three, which consists of 53 extended NetFlow features.

After thorough review of all five datasets, it was clear that the NF-UNSW-NB15 dataset provided the best balance of benign and malicious traffic, had enough samples for the models to generalize learnings, maintained meaningful network structure, and had multi-class labels for attacks.

3.3.2 Embedding Model

This report distinguishes between architectural hyperparameters (e.g. number of layers, nodes per layer, activation functions) and training hyperparameters (e.g. learning rate, batch size). This is important as the model architecture greatly affects the training hyperparameters and should be static before further tuning. To evaluate the efficacy of each instance of the embedding model, two dimensionality reduction techniques were used.

Principal Component Analysis (PCA) (Pearson, 1901; Hotelling, 1933) is a technique that linearly transforms high-dimensional data into n principal components. These principal components are calculated by finding orthogonal directions in the data that maximize variance and extracting the top n with the highest variance. This allows the writer to visualize the high dimensional data in two- or three-dimensional plots. Since this is a purely linear transformation, it is useful to visualize the global structure of the embeddings.

t-Distributed Stochastic Neighbor Embedding (t-SNE) (van der Maaten and Hinton, 2008) is another technique that preserves local similarities in high-dimensional data. This is helpful to visualize local clusters of high-dimensional data and captures complex, non-linear relationships that PCA misses. Since this algorithm is considerably slower than PCA, the dataset should be first pre-processed with PCA to a lower dimensionality and then processed with t-SNE. It is also important to note that t-SNE does not preserve global structure, so distances between clusters are not necessarily accurate.

3.3.2.1 Architectural Hyperparameters

Figure 3.3-2 shows the initial architecture defined for the embedding model. This takes a batch tensor with each sample being 34 values, 32 for each bit of the source IP and one for protocol and port (normalized to floats between 0 and 1). The batch then gets sliced into the three categories and projected to larger dimensions to be concatenated again. Once the batch is at the desired embedding dimension (150-d in this case) it is

propagated through an identically sized linear layer, with no activation function, to produce the final embedding. This allows the model a latent space to generalize and mix signals from the projected categories.

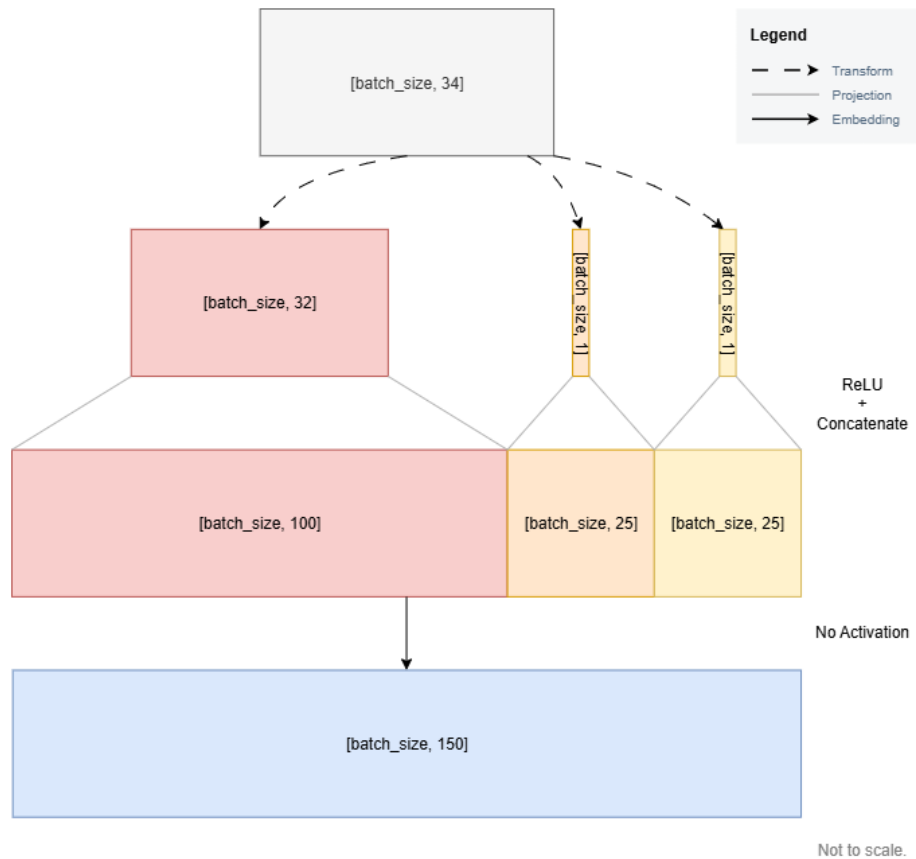


Figure 3.3-2: Initial Model Architecture

As per the evaluation of this architecture in Section 4.2, the PCA-2 plots showed that the model had learned a degenerate embedding. This was due to the port and protocol fields being assigned a value from zero to one. Since larger port and protocol sample values were numerically larger and only the source and destination IPs were used for context selection, the model was learning to organize the samples from smallest to largest port and protocol. This caused the semi-anisotropic structure shown in Figure 4.2-1. Due to this, the model architecture was changed to use small embedding layers to project the port and protocol fields. In addition, the context selection criteria were changed to also

classify samples with the same destination port or the same protocol as positive context items. See Figure 3.3-3 for the corrected model architecture.

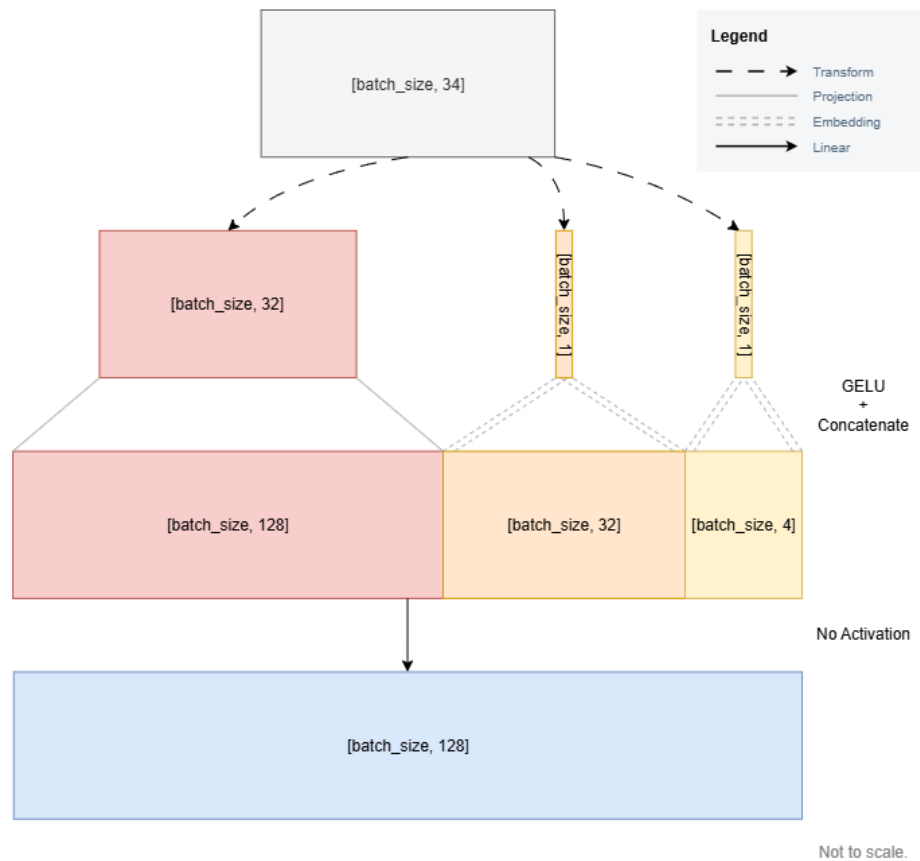


Figure 3.3-3: Corrected Model Architecture

The final change to the architecture was adding one more hidden layer twice the size of the output layer (referred to as the fusion layer). This affords the model more weights to learn more complex relationships, with the compression between the fusion layer and output layer forcing the model to make embeddings denser and keep only the most important information. See Figure 3.3-4 for the final architecture.

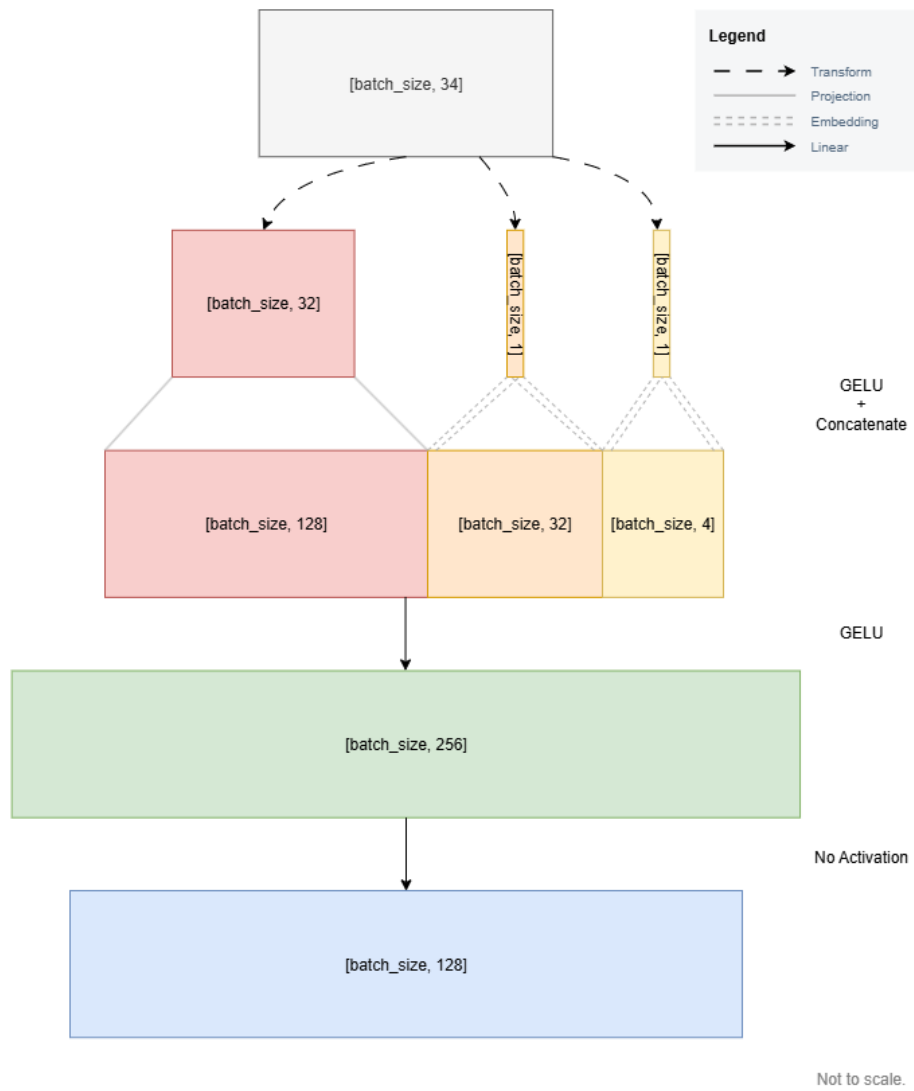


Figure 3.3-4: Final Model Architecture

With these changes, the global structure of the embeddings clustered into clearer and more varied clouds. See Figure 4.2-4 for the PCA-2 plots of the corrected model architecture. Figure 4.2-5 shows the t-SNE-3 projection of the embeddings, which demonstrates that the subnets are still clustering in a complex non-linear manner which was not clear when observing only the PCA plots.

Additionally, to improve clustering, the activation function was changed to GELU instead of ReLU. GELU was introduced by Hendrycks and Gimpel (2023), as an alternative to

ReLU and ELU. They concluded that GELU outperformed both functions in a variety of tasks, including classification and autoencoding. Although the paper does not describe performance for the exact model architecture seen in this report, the IP2Vec architecture can be likened to an autoencoder without a decoding head. See Figure 3.3-4 for the autoencoding performance benchmarks shown by Hendrycks and Gimpel. Each curve is the median of three runs. Light, thin curves correspond to test set log losses.

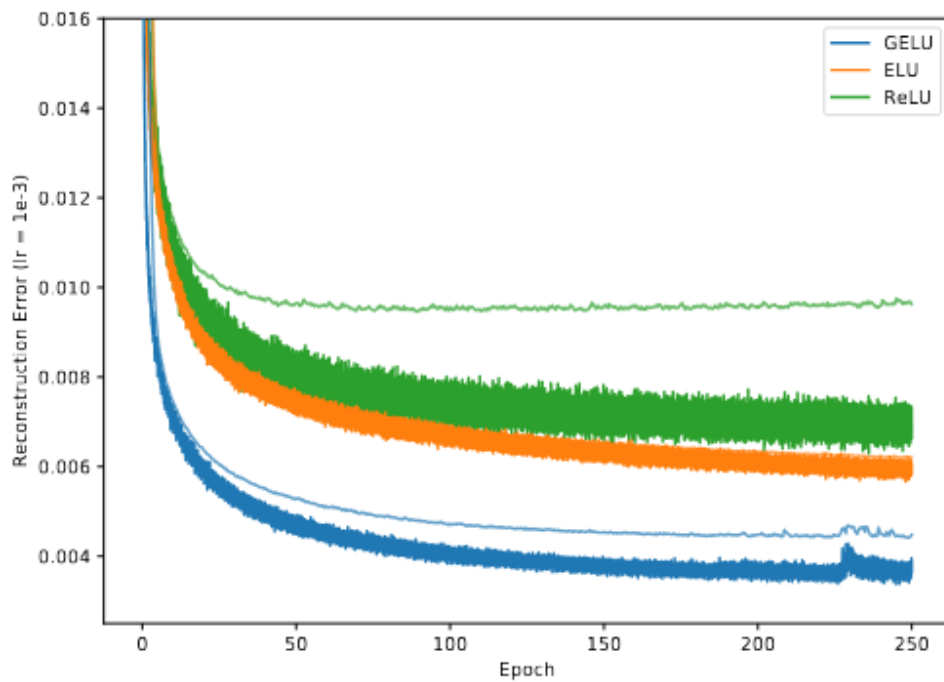


Figure 3.3-5: MNIST Autoencoding Results (Hendrycks and Gimpel, 2023)

3.3.2.2 Training Hyperparameters

The training hyperparameters were mostly taken directly from the *Burn* framework's exposed tuneable parameters. These include training epochs, batch size, split ratio, concurrent threads, and learning rate. These are standard hyperparameters used in most ML projects and can be adjusted to balance performance and embedding quality. In addition to these, there are two implementation-specific parameters available to use: context window, and negative multiplier. These are explained in further detail in section 3.4.1.2, but their main use is to change the number of contexts to evaluate the correctness of a sample embedding. With a larger context window, the model can learn

to move embeddings closer to similar network entities, and with a larger negative multiplier, the model can learn to move embeddings farther from dissimilar network entities.

3.3.2.3 *Command Line Interface*

The embedding model contains two binaries: training and inference. The training binary requires the CSV column name for each of the utilized features: source IP, destination IP, destination port, and protocol. This is because this project utilizes a custom deserializer implementation for dynamic dataset feature names, which allows for any dataset to be used with the produced artefact. The requirement can be forgone if a “store” argument is provided. This argument is used to provide a folder path to configurations which were generated from a previous training. In addition to these arguments, there are six learning hyperparameters which can be adjusted from the CLI. Note that these hyperparameters are saved in the store path, but they are overwritten by any further CLI arguments used in any subsequent runs. The exposed hyperparameters are as follows:

- Epochs to train for
- Percentage of dataset to use for training vs. validation
- Batch size
- Learning rate for gradient adjustment
- Amount of context examples per sample
- Amount of negative context example per positive context.

Finally, there are two meta parameters which have no effect on training or model performance. These are the RNG seed (for reproducibility), which is set to one for all experiments in this report, and CPU threads (for dataset loading/batching and metrics).

The inference binary makes use of subcommands for more fluid ergonomics and conditionally gated arguments. All inference runs require a store argument, since they require a trained model to work. The CLI gives the user an option of single sample

inference or batched inference from a CSV file. For single samples, the source IP, destination port, and protocol need to be passed as arguments. For batched inference, only a file path is required. Finally, for both types of inference, PCA and t-SNE reduction is available through their respective subcommands. PCA reduction only takes one positional argument and two optional flags. The positional argument is the number of dimensions to reduce to, and the optional flags are used to determine whether the PCA fitting should be saved to the store or loaded from the store (mutually exclusive). t-SNE reduction takes the same target dimension argument, but requires a theta parameter, which is used for the Barnes-Hut estimation of the t-SNE algorithm. A larger theta lowers accuracy but lowers runtime, whereas a smaller theta increases accuracy but increases runtime.

3.3.3 Classification Models

The codebase of the classification models is separated into three main modules: dataset, models, and metrics. The parent dataset module contains all the data structures used to load and use the dataset. The dataset module contains various submodules for feature selection (scoring and selection) and dataset transformers. These submodules are all brought together to be used in a 'Pipeline'. This defines the main preprocessing process and is made up of three stages: filtering, selection, and transformation. The 'models' module contains one sub-module for every implemented model type. Finally, the metrics module implements all metric measurements for any generic output in conjunction with a 'Dataset' object.

3.3.3.1 Dataset

This implementation uses two main data structures for representing a dataset. The first is a dataset schema, which is loaded from a TOML file. This file simply lists each feature from the dataset and assigns it a string designating its type. These strings are "cat" for categorical features, "num" for numerical features, "label" for the target labels, and "ignore" for ignored features. This TOML file is parsed with Serde, and each feature is mapped to a 'FieldKind' enum, which is either 'Feature', 'Label', or 'Ignored'. The

‘Feature’ variant contains a ‘Feature’ struct which has two fields: ‘kind’ and ‘col_idx’. The ‘kind’ field uses a ‘FeatureKind’ enum, which identifies the feature as either ‘Numeric’ or ‘Categorical’. The ‘col_idx’ field designates which column the feature will occupy on the sample matrix in the Dataset. Having this stored in the schema ensures that multiple dataset objects can guarantee the same features are stored in the same columns. This split between “fields” and “features” allows the dataset to be able to query which features are numerical or categorical and ensure only the ‘Feature’ variants have column indices, as the label and ignored fields will obviously not have any column in the sample matrix.

Utilizing the ‘DatasetLoader’ struct, a ‘Dataset’ object is created from the path to the actual dataset, the path to the generated IP embeddings, and a ‘DatasetSchema’. See below for the fields of the ‘Dataset’ struct.

```
/// Struct containing dataset samples, labels, and schema
#[derive(Debug, Clone)]
pub struct Dataset {
    /// [Array2] of shape [n_samples, n_features]
    pub samples: Array2<f32>,
    /// [Array1] of shape [n_samples]
    pub labels: Array1<usize>,
    /// [Vec] of length [n_classes]
    pub label_strings: Vec<String>,

    /// [Array2] of shape [n_samples, embedding_dim]
    pub embeddings: Array2<f32>,

    /// Metadata
    pub schema: Arc<DatasetSchema>,
}
```

As can be seen, the IP embeddings are stored separately to the samples, since they are exempted from any dataset preprocessing, which would lose the dense information contained in each vector.

For dataset preprocessing, the feature selection module contains all the scorers within the scoring submodule. This submodule defines a generic ‘ScoreFeatures’ trait which defines a ‘score’ method, that takes a dataset and returns a ‘Vec’ of ‘FeatureScore’. ‘FeatureScore’ is a simple struct which contains the ‘DatasetSchema’ field index of each scored feature and a float score. The ‘score’ method does not take any field indices as

each scorer automatically selects compatible features from the dataset. The selection submodule contains a 'SelectionStrategy' enum which contains three different strategies: top-k, threshold, and percentile. These each have a value which is used to select features based on their FeatureScore. The enum has a 'select' method which takes a slice of feature scores and returns an array of field indices of features that were selected. The dataset transformers have two generic traits, 'Fit' and 'Transform'. These utilize a common Rust pattern called the type-state pattern. This adds a generic parameter to each transformer struct and only implements the 'Fit' trait on an 'Unfitted' transformer and the 'Train' trait on a 'Fitted' transformer. Splitting the split and transform functionality prevents data leakage across the train and valid sets, since the transformers are fitted on the training data and can transform any other dataset such as the validation set or any future production datasets.

3.3.3.2 Models

To ensure models are interchangeable, two generic traits are implemented on each one. The 'Train' trait defines a 'train' method which takes a dataset and returns a trained model. The trained model is defined in an associated type. All models utilize the type-state pattern to ensure that no user can call the 'infer' method before training. The type-state pattern adds a generic parameter which is a simple zero sized type (ZST) marker named 'Trained' and 'Untrained'. Since the constructor is only implemented on 'Untrained', the user does not see these types at all in usage, and it does not affect the API ergonomics. The 'Infer' trait defines the 'infer' method and is only implemented on the 'Fitted' model, this method returns a 1D array of integers, which identify which class has been predicted by the model.

3.3.3.3 Metrics

Since all calculated metrics rely on the number of true positives (TPs), false positives (FPs), true negatives (TNs), or false negatives (FNs), all metric calculations are implemented within a 'ConfusionMatrix' struct which holds the complete matrix, label strings (collected from the dataset), and an array for each prediction outcome. All of this

can be printed to the terminal through a custom Debug implementation for the ‘ConfusionMatrix’ struct. Additionally, the ‘ConfusionMatrix’ struct has methods for calculating accuracy, recall, specificity, precision, and negative predictive value.

3.4 Development

3.4.1 Embedding Model

3.4.1.1 Deserialization

For the dynamic column name deserialization, both the CSV reader object and the column name strings are passed to the function. The column name strings are taken from the CLI arguments detailed in Section 3.3.2.3; these are mapped to a ‘FieldKind’ enum which helps pattern match inside the deserialization hot loop. For performance, a header map is constructed, which maps the column index of each header (matching to the field map) to one of the four features. The result is a map of each relevant column index to an enum variant. The hot loop simply needs to visit each index and assign each sample feature value. If one of the values is missing for any record, an error with the feature name is thrown. With this sample data, an ‘Ip2VecDataset’ struct is created. However, the dataset does not simply store an array of ‘Sample’ objects containing each feature by value. Instead, the dataset allocates an Atomically Reference Counted (Arc) pointer for each feature value and stores each host, port, and protocol pointer in a separate HashMap. This ensures each item is only stored once. Additionally, a sample is constructed with the same pointers and pushed to a Vec. Arcs count each reference to the data, and keep the data allocated until the count reaches zero, when all the references are dropped. This ensures that each host, port, and protocol is only stored once in memory, and even if a sample is pruned from the dataset, the data survives (if another sample is referencing it).

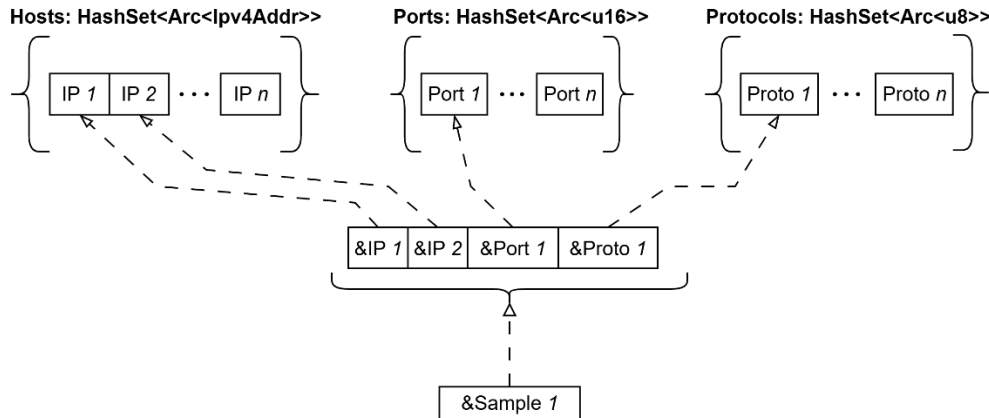


Figure 3.4-1: Sample Design Architecture

3.4.1.2 Preprocessing

In this implementation, preprocessing is a separate procedure. Initially the ‘context’ field of the ‘Ip2VecDataset’ struct is None, and it is populated only after calling the ‘preprocess’ method. This method takes a random number generator (RNG), a positive context window, and a negative context multiplier. The contexts field is an optional HashMap of Arc pointers. The positive contexts for each sample are simply collected through the following iterator chain:

```
let positive: HashSet<Arc<Sample>> = self
    .samples
    .iter()
    .filter(|other| sample.is_context(other))
    .cloned()
    .take(context_window)
    .collect();
```

The ‘is_context’ method checks for three conditions: if the other sample’s source IP is the same as the current sample’s destination IP, if the other sample’s destination port is the same, or if the other sample’s protocol is the same. This context selection method is similar to one proposed by Ring *et al.* in their IP2Vec paper (2017). Figure 3.4-2 shows their approach to generating training samples (context).

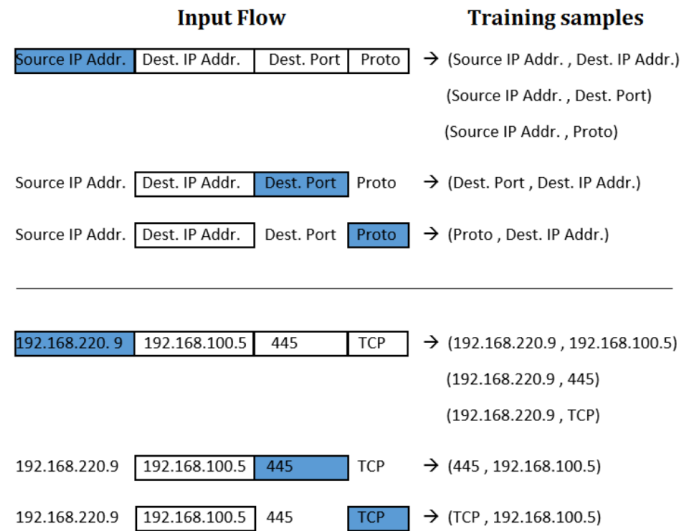


Figure 3.4-2: Generation of training samples (Ring et al., 2017)

This figure depicts input words (target) in black squares with a blue background and context words in black squares with a white background. Ring et al. note (2017, s. 4.C):

We do not use the Destination IP Address as input word since we build upon unidirectional flow-based data. Responses from the Destination IP Address are captured in a separate flow where the roles of Source IP Address and Destination IP Address are swapped (see flow #1 and #2 in Table I). Therefore, considering both IP Addresses as input words would create duplicate training samples.

Since this implementation uses the entire “entity” as a target (including port and protocol), instead of matching individual hosts to ports and protocols, the system considers any sample with matching features as a context. This means that the neural network itself will learn relationships between network entities, rather than individual features.

For negative samples, the ‘preprocess’ method does not use an iterator chain with the *rand* library’s sample iterator adapter. This is because *rand* uses population sampling, which is a greedy algorithm and needs to iterate through all the samples per sample (for a complexity of $O(n^2)$). This cannot be buffered before the sample loop, as each sample needs to filter for non-contextual samples; therefore, the following loop is used.

```

while negative.len() != (context_window * neg_multiplier) {
  let i = rng.random_range(0..self.samples.len());
  let other = &self.samples[i];

  if !sample.is_context(other) {
    negative.insert(Arc::clone(other));
  }
}

```

This ensures that each sample only iterates the minimum number of times (plus collisions with positive context samples).

Finally, the last step of preprocessing is to prune samples which do not have enough context. This architecture avoids removing feature data which other samples depend on through its Arc pointer HashMap design. This means samples with not enough context can be simply removed from the samples Vec without any side effects.

3.4.1.3 *Batching*

For proper training, the dataset needs to be batched. This allows the optimizer to perform small gradual updates to the neural network’s weights and biases. When using the *Burn* framework for deep learning, the framework handles batch sizing through a ‘Dataset’ trait method which gets individual samples. A separate ‘Batcher’ trait is implemented on the ‘ContextBatcher’ struct, which takes an array of ‘ContextItem’ and tensorizes the data. This is illustrated in Appendix A, which shows the data flow from the ‘Ip2VecDataset’ object to a single ‘ContextBatch’. The tensorization is implemented at the sample level, through the ‘encode’ method shown below.

```

pub fn encode(&self) -> Vec<f32> {
  let mut data: Vec<f32> = Vec::with_capacity(34);

  let src_ip = self.src_ip.to_bits();

  for i in 0..32 {
    data.push(((src_ip >> i) & 1) as f32);
  }

  data.push(*self.dst_port as f32 / 65535.0);
  data.push(*self.protocol as f32 / 255.0);

  data
}

```

Each sample tensor is made up of 32 elements representing the source IP of the sample, plus two bits representing the destination port and protocol. The ‘to_bits’ method returns

a u32 integer, representing the IP in the compiling platform’s endianness. The for-loop bit-shifts the u32 and performs a bitwise AND on the current bit to determine if it is set or not, pushing the bit to the tensor data.

For performance reasons, all tensors are built by first buffering the data contiguously in a one-dimensional Vec and then building the Tensors with their respective shapes. The shapes are defined as follows.

```
let sample_dims = [batch_size, encoded_dim];  
let pos_context_dims = [batch_size, positive_count, encoded_dim];  
let neg_context_dims = [batch_size, negative_count, encoded_dim];
```

This tensorization procedure avoids repeatedly stacking Tensors, which is a costly operation that repeatedly allocates and assigns a new Tensor in the GPU memory for each sample.

3.4.1.4 Forward Pass

The *Burn* framework standardizes the construction of the NN layers by ensuring every field of a NN module derives the Module trait. The forward pass of the embedding network consists of two methods which differ in scope. The ‘embed’ method embeds a single two-dimensional tensor of shape $[b, 34]$, where b is the batch size. This follows the method set out in Section 3.3.2.1; the key detail is that the two port and protocol neurons are scaled back to their original sizes (65,336 for port, and 256 for protocol) and casted to integers. This is because the embedding layers require integer indices to embed the values. The reason for the utilization of embedding layers instead of regular linear layers is discussed in depth in Section 3.3.2.1. The projected features are then concatenated and passed through the fusion and compression layers. The output of the compression layer is L2 normalized to maintain the embeddings in a unit hypersphere and avoid lossy normalization in the downstream classifiers. This means the resultant embeddings can be treated separately to the other dataset features and easily concatenated after dataset preprocessing.

The ‘forward’ method is utilized only in training, and it takes all the fields of the ‘ContextBatch’ defined in the previous section. The targets are embedded with the ‘embed’ method mentioned previously. The positive and negative context samples need to be flattened to two dimensions by squeezing the batch size and context window dimensions before embedding. The returned embeddings are then expanded back to three dimensions for the loss calculation.

3.4.1.5 Loss Function

The objective function selected for learning the complex relationships between embeddings is borrowed from the original Word2Vec paper published by Mikolov *et al.* (2013). In this paper, they discuss two different learning objectives used for the skip-gram learning task. The skip-gram learning task is defined by trying to maximize the probability that each target word predicts its context words. The learning objectives are the loss functions used to calculate the success of each prediction. The objectives proposed are hierarchical softmax and negative sampling. Hierarchical softmax uses a binary tree to model a true full softmax function (which naively frequency encodes all possible outputs). This is used to improve performance of the softmax objective. Negative sampling (often called NEG loss) changes the objective completely and reframes the learning problem to simple logistic regression. This is achieved by scoring positive context pairs as similar and negative context pairs as dissimilar. This does not provide an accurate probability distribution, but for relational embeddings (like the ones implemented in this report) this is not necessary. This model does not define a discrete output space and therefore relying on a softmax-based categorical likelihood becomes highly complex and restrictive. Instead, this model behaves like a contrastive learning model. While Noise Contrastive Estimation (NCE) is often used for problems like these, Mikolov *et al.* (2013) introduce NEG, inspired by NCE, yet avoiding calculating a probability distribution, making it significantly more computationally efficient and fit-for-purpose in this case.

This implementation further tweaks NEG loss by using multiple positive contexts per target. Given x_t is the target sample, c is the context window, k is the negative context multiplier, $p_{i,t}$ is a positive context sample, and $n_{i,t}$ is a negative context sample:

$$NEG(x_t, p_t, n_t) = \sum_i^c \log \sigma(p_{i,t} \top x_t) + \sum_i^{c \times k} \log \sigma(-n_{i,t} \top x_t).$$

The only difference between this objective and the original NEG loss is the summation of the positive log sigmoid term, as the original objective does not account for multiple positive contexts.

The *Burn* framework includes various implemented loss objectives; however, there is no NEG loss implementation. Thankfully, implementing a custom loss function is simple, yet not extensively documented. The bulk of the implementation lies in the ‘forward_no_reduction’ method shown below.

```
let unsqueezed_target = target.unsqueeze_dim(1);

let positive_similarity: Tensor<B, 1> = unsqueezed_target
  .clone()
  .matmul(positive.swap_dims(1, 2)) // [batch_size, 1, context_window]
  .squeeze_dim::<2>(1) // [batch_size, context_window]
  .sum_dim(1) // [batch_size, 1]
  .squeeze_dim::<1>(1); // [batch_size]

let negative_similarity: Tensor<B, 1> = unsqueezed_target
  .matmul(negative.swap_dims(1, 2)) // [batch_size, 1, context_window * neg_mult]
  .squeeze_dim::<2>(1) // [batch_size, context_window * neg_mult]
  .sum_dim(1) // [batch_size, 1]
  .squeeze_dim::<1>(1); // [batch_size]

(log_sigmoid(positive_similarity) + log_sigmoid(-negative_similarity)).neg()
```

The above code is annotated with the resultant tensor’s shape. Note that since the target tensor is two-dimensional, it must be unsqueezed (adding a size one dimension in the second dimension) to be compatible to matrix multiply with the three-dimensional context tensors. The context tensors must also swap their first and second dimensions to match their second-to-last dimension to the target’s last dimension. After this, the broadcast second dimension is squeezed out, and the context dot products are summed. The trailing dimension is redundant, as each row has identical elements, thus it is squeezed out. Finally, the log sigmoid functions are applied to both the positive

and negative similarity scores (with the negative being inverted) and their sum is negated, as the NEG objective function defines a value to be maximized, and *Burn* requires loss as a value to be minimized.

3.4.1.6 Dual GPU Integration

The IP2Vec training binary supports dual GPUs for maximizing training performance, *Burn* supports using an arbitrary number of GPUs for the train stage. Once a train step is completed and the gradients are updated, the tensors are synced to the CPU for calculating loss and metrics. This behaviour is toggled through the “IP2VEC_DUAL_GPU” environment variable which is detected with the following code.

```
static TRAINING_DEVICES: LazyLock<Vec<LibTorchDevice>> = LazyLock::new(|| {  
    let mut devices = vec![LibTorchDevice::Cuda(0)];  
  
    if std::env::var("IP2VEC_DUAL_GPU").is_ok() {  
        devices.push(LibTorchDevice::Cuda(1))  
    }  
  
    devices  
});
```

This variable is static as it only needs to be initialized once for the entire program’s lifetime. Furthermore, LazyLock ensures that the environment variable can be read at runtime, rather than compile-time. Additionally, it automatically implements deferred initialization, whilst circumventing the static keyword’s const requirement and ensures thread-safety. This variable is used in the ‘train’ method to declare the ‘MultiDeviceNaive’ learning strategy. This enables *Burn* to create and use tensors on both GPUs.

3.4.2 Classification Models

3.4.2.1 Dataset

The ‘DatasetSchema’ simply reads all the fields in the schema TOML file and creates the schema structure from the kind strings as described in section 3.3.3.1. Dataset loading was implemented similarly to the embedding model. A field map was created from the ‘DatasetSchema’, excluding ignored fields. Then, when processing individual CSV records (rows), only the necessary column indices were iterated through, and the sample

and label buffer were populated. Embeddings were simpler, as each entire row constituted an embedding. Then, the matrices were constructed from shapes computed from the counted number of samples and the total buffer size.

Once a dataset is constructed, there are several utility functions which can be applied to the dataset. These are: ‘select_features’, ‘split_with_ratio’, and ‘shuffle’, among others. The ‘select_features’ method is used by the dataset pipeline to select features from the array of selected indices returned by a ‘SelectionStrategy’. This method returns a new dataset with only the selected features. The ‘split_with_ratio’ returns a tuple of two datasets with the first dataset having ratio% of samples and the second dataset having the rest of the samples. This is useful for generating a train test dataset split. Finally, the ‘shuffle’ method, which shuffles the indices and applies the new indices to the samples, embeddings, and labels. This allows the dataset to maintain the class ratios across the training and validation sets.

For dataset preprocessing, there are four feature scorers that have been implemented. These are: variance, sparsity, ANOVA F-score, and Chi-Squared. Variance and sparsity are mainly used as filter selectors, and apply to all features. Variance applies to all features and is calculated with the following formula. Given c is the column, $\bar{c}$ is the column mean, x is a value in the column, and n is the total number of samples.

$$V = \sum_c^x (x - \bar{c})^2 \div (n - 1)$$

Sparsity is simply defined as the count of non-zero values divided by the total number of samples in the feature column.

Next, ANOVA F-score and Chi-Squared are used for scoring numerical and categorical features, respectively. These were introduced to the field of ML by Guyon and Elisseeff (2003). The ANOVA F-score of feature i is defined as follows.

$$F_i = \frac{v_{between}}{v_{within}}$$

Where $v_{between}$ is the variance of each feature between classes and v_{within} is the variance of each feature within each class. The variance between classes is calculated by grouping each feature sample by class, summing the square difference between each class mean and the overall feature mean. The variance within classes is calculated by grouping each feature sample by class and summing the square difference between each class sample and the class mean. This scoring algorithm can be interpreted quite simply, a higher F-score indicates that the numerical feature values have a high correlation with the class label, and a low F-score indicates that the feature values have little effect on the class of the sample.

The Chi-Squared test is used to score categorical features, since it utilizes the counts of each value rather than a numerical value. The algorithm for feature i is defined as follows.

$$\chi_i^2 = \sum_{j,k} \frac{(O_{jk} - E_{jk})^2}{E_{jk}}$$

Where j is the current feature value, k is the current class, O is the observed count of each feature value, and E is the expected count of each feature value. The expected count is a simple calculation, for each feature and class pair, the count of value j multiplied by the count of class k divided by the total number of samples. The computed score can be interpreted similarly to ANOVA F-scores, a higher value means the feature is more statistically relevant to the class label. This means a feature with higher score will be more useful in predicting the class label.

The implementation of feature selection strategies is generally self-explanatory. The threshold strategy simply selects features that have a score higher than the threshold. The top-k strategy sorts the feature scores and selects the top k features. And finally, the percentile strategy sorts the feature scores and selects the top percentage of features, which is defined as part of the enum variant.

There are three transformers available to use: a frequency encoder, a standard scaler, and a robust scaler. The frequency encoder iterates through all the values of a categorical feature column and either increments or adds a new count to a HashMap mapping each value to a count. Then, when transforming, the value is simply looked up on the stateful HashMap and the retrieved count is scaled by the number of samples. The standard scaler scales numerical values utilizing the column mean and standard deviation. The standard scaling algorithm can be described with the following formula.

$$x' = \frac{x - \bar{c}}{\sigma}$$

Where x is a value in column c , $\bar{c}$ is the column mean, σ is the column's standard deviation, and x' is the resultant value. This transforms the data to have a mean of 0 and a standard deviation of 1. Robust scaling is a statistical function which uses the interquartile range and the median instead of the standard deviation and mean. This makes scaled values robust to outliers, which is important for anomaly detection. The robust scaling formula is shown below.

$$x' = \frac{x - \tilde{x}}{IQR(c)}$$

The IQR represents a function calculating the interquartile range of the column, which is calculated by subtracting the 75th quartile from the 25th quartile.

Lastly, all these preprocessing steps are pulled together with the 'Pipeline' struct. Since the 'Fit' trait requires an associated type, this causes problems with storing heap-allocated boxes with dynamically sized types. To remedy this, a 'FitDyn' adapter trait was implemented as below.

```

trait FitDyn {
  fn fit_dyn(&self, dataset: &Dataset) -> Result<Box<dyn Transform>>;
}

impl<T> FitDyn for T
where
  T: Fit + 'static,
  T::Fitted: Transform + 'static,
{
  fn fit_dyn(&self, dataset: &Dataset) -> Result<Box<dyn Transform>> {
    Ok(Box::new(self.fit(dataset)?))
  }
}

```

This blanket implementation allows the ‘Pipeline’ struct to store boxes of ‘dyn FitDyn’ and store any of the three available transformers in the same array. The struct contains three methods to add filters, selectors, and transformers. These allow the end-user to just pass the transformer, and the methods will automatically box the objects and add them to their respective array. The ‘run’ method first performs a train test split with an 80:20 ratio. Then, it sequentially applies the filters and selectors. For both of these, the method scores the features on only the training set and selects the field indices on both the training and test sets. After the dataset features have been reduced, the transformers are fitted on the training set, and the fitted transformer is used to transform both the train and test sets. With this implementation, the usage of the preprocessing pipeline looks like below.

```

let pipeline = Pipeline::new()
  .add_selector(Anova::default(), SelectionStrategy::TopK(5))
  .add_selector(ChiSquared::default(), SelectionStrategy::TopK(5))
  .add_transformer(RobustScaler::new())
  .add_transformer(FrequencyEncoder::new());

let (train, test) = pipeline.run(dataset)?;

```

3.4.2.2 Models

At this point, the development of the tool was impeded by performance issues when training models. Using *SmartCore*, a linear SVM model took more than 24 hours to train, even with a reduced embedding dimension of 32. For testing purposes, despite the results shown in section 4.1, a *Linfa* SVM was also implemented. This proved to be worse, as the *Linfa* implementation of SVM attempted to compute the entire gram table of the

training set and store it in memory, which proved to be 14.3TB of memory. These issues could not have been foreseen with the preliminary experiments described in section 3.2, since these issues arose only when testing a large-scale dataset like NF-UNSW-NB15-v3. Because of these issues, the developer elected to use bespoke C++ libraries with Rust bindings for each classification algorithm, as these are more robust and have been more thoroughly tested with larger datasets. Two algorithms were chosen for this dissertation: support vector machines (SVM), and gradient boosted decision trees (GBDT).

The SVM algorithm attempts to find a hyperplane for each class that maximizes the margin between classes. This can be simply adapted to a multi-class problem by utilizing the One vs. Rest method. This trains k classifiers for each class, and each classifier attempts to maximize the margin between its class and all others. In this implementation, a linear solver was used, this means that the dividing hyperplanes will be linear and will struggle to separate non-linear data. This decision was taken since non-linear kernel SVMs have a time complexity of around $O(n^3)$ and a memory complexity of $O(n^2)$ which make them unsuitable for such a large dataset. A linear kernel allows the SVM model to be trained within a reasonable amount of time, making it practical for real-world usage. As discussed in section 3.1.4, the library chosen was *LIBLINEAR*, as it is the large-scale implementation of *LIBSVM*. The *liblinear* Rust bindings have a crucial implementation detail, which is that the model struct is not made public, thus it is not made available to the library's users. This is important because the 'Trained' state needs to be able to hold the classifier to be accessible from the inference function. To surmount this issue, an 'Adapter' trait was implemented for all objects that implement the 'LibLinearModel' trait, which is how the library makes the model methods available. This adapter simply takes features and returns a probability output, essentially delegating the function. This allows the developer to store a heap-allocated Box with a dynamic type of objects implementing the 'Adapter' trait. Another interesting detail is that when converting the dataset from the 'ndarray' Array2 to a 2D array, the 'from_dense_features' method contained a bug. This bug took the length of the outer array as the number of

features, rather than the length of the inner array. This was not consistent with the rest of the bindings, which expected everything else in row-major order. To fix this, the data needed to be built from sparse features, and the lower-level node mapping required by the C++ library needed to be built manually with the following code.

```
let nodes: Vec<Vec<(u32, f64)>> = features
  .mapv(|v| v as f64)
  .rows()
  .into_iter()
  .map(|row| {
    row
      .into_iter()
      .enumerate()
      .map(|(i, v)| ((i + 1) as u32, *v))
      .collect()
  })
  .collect();
```

The ‘liblinear’ library exposes inference functionality with a ‘predict’ method that only takes one sample at a time. This means that the samples rows need to be iterated through and the predict function must be called once for each sample.

The GBDT library chosen was *XGBoost* (Chen and Guestrin, 2016), with the *xgb* bindings. For the individual tree parameters, there were four parameters set: ETA (also known as learning rate), max depth, min child weight, and max delta step. The ETA was set to 0.1, which is the standard for most models. The other three parameters were informed by the *Notes on Parameter Tuning* page of the *XGBoost* official documentation (DMLC, 2025). According to this documentation, the max depth and max delta step parameters control model complexity, which helps to reduce the risk of overfitting trees. The max depth was set to 6, and the min child weight was set to 1. The max depth value allows the trees to split up to 6 times per path, allowing the tree to model complex non-linear relationships while avoiding overfitting. This value is on the lower end of the common range, as these parameters apply only individual trees and higher-order non-linearities can be captured with the sequential boosting structure that GBDTs use. Conversely, a min child weight of 1 is generally considered quite permissive and allows models to detect subtle patterns. This was done because the max depth parameter provides a strong guardrail against overfitting, and the min child weight parameter restores fine-grained expressivity to each tree. Finally, the max delta step parameter was introduced to help with the heavily

imbalanced dataset, common in IDS scenarios. This constrains the tree updates when very large gradients (caused by heavy imbalance at splits) to prevent aggressive updates which can lead to unstable training curves. A further measure to remediate the class imbalance was to add inverse class weights to the dataset through the ‘set_weights’ method on the training matrix. These were calculated using the following code.

```
let class_counts = dataset.get_class_counts();
let total: usize = class_counts.iter().sum();
let weights: Vec<f32> = dataset
    .labels
    .iter()
    .map(|&l| total as f32 / (class_counts.len() as f32 * class_counts[l] as f32))
    .collect();
```

GBDTs natively support multi-class classification, so no One vs. Rest or One vs. One ensemble was necessary. A bug in the ‘TrainingParametersBuilder’ within the *xgb* bindings prevented the booster classifier from creating any trees, no real cause was found by the developer, but this was overcome by manually setting training parameter strings and manually iterating through updates. The training iterations was set to 100, which allowed the model to reach a local loss minimum.

After initial observations were made, it was clear that the class imbalance issue was still affecting training of the GBDT model, so a variant was developed. This model was called the cascading GBDT model. The idea behind this change was to perform a coarse binary classification on benign vs. anomalous data, and then a fine-grained classification of malicious class types. There were two strategies implemented for the fine-grained classifier. Firstly, the native multi-class objective function could be used with a single classifier to classify the malicious samples. Otherwise, a One vs. Rest approach could be utilized to train k binary classifiers and perform majority voting to classify samples. To implement the cascading classifier, the ‘Dataset’ struct needed some additional utility functions, namely: negative and positive binary labelling and malicious-only sample segregation (for training). Negative binary labelling took a label argument and replaced all the matching labels and converted them to zero, whilst setting all non-matching labels to one. Positive binary labelling did the opposite, setting the matching labels to one and the non-matching to zero. The negative binary labels were used for the coarse

classifier, and the positive binary labels were used to generate labels for each malicious binary classifier. This is implemented in the same way to the normal GBDT classifier, with the only difference being that the fine-grained classifier is only trained on malicious data. At inference, the fine-grained classifier is not guaranteed to get only malicious data, so the inference method simply classifies anything the coarse classifier deems malicious. For the One vs. Rest strategy, the majority voting is performed with the following code.

```
class_outputs
  .axis_iter(Axis(1))
  .map(|column| {
    column
      .iter()
      .enumerate()
      .max_by(|(_, a), (_, b)| a.partial_cmp(b).unwrap())
      .context("not able to find maximum for OvR models")
      .unwrap()
      .0
  })
  .collect::<Array1<usize>>()
```

3.4.2.3 Metrics

As described in the Design Structure section, most metrics are implemented on the ‘ConfusionMatrix’ struct. The matrix itself is constructed with the following loop.

```
for (actual, predicted) in self.labels.iter().zip(output) {
  confusion[(*actual, *predicted)] += 1;
}
```

Then, the prediction outcomes are stored in four arrays, with each index corresponding to an output class. The true positives are simply found at the diagonal of the confusion matrix. The false positives are calculated by adding the counts of each value of the current class’s column where the row index is not equal to the class label. The false negatives are calculated with a similar iterator chain, which adds all the counts where the column index is not equal to the current class label. Finally, the true negatives simply subtract the true positives, false positives, and false negatives from the total sum of the confusion matrix. The confusion matrix metrics are calculated with the following formulae, where TP is the true positives, TN is the true negatives, FP is the false positives, and FN is the false negatives.

$$Recall = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

$$Precision = \frac{TP}{TP + FP}$$

$$NPV = \frac{TN}{TN + FN}$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

It can be difficult to balance these performance indicators when evaluating a model, thus, in the ‘Classification Metrics’ chapter of *Machine Learning Algorithms* (2018) Bonaccorso highlights two common functions to evaluate overall performance more precisely. The first of these functions is called the F_β score, given an input value of β , a precision value of p , and a recall value of r :

$$F_\beta = (\beta^2 + 1) \times \frac{p \times r}{\beta^2 p + r}.$$

This is described by Bonaccorso as a “weighted harmonic mean between precision and recall”. When the β value is 1, the F_1 score equally values precision and recall. As β is increased beyond 1, the output values recall more; conversely, as β is decreased, the output values precision more. Therefore, F_β is useful to interpret overall accuracy as a trade-off between precision and recall.

The second performance evaluation technique is the ROC (receiver operating characteristics) curve. This plots false positive rates against true positive rates across a range of classification thresholds. The larger the area under the curve, the better the model in question is performing. However, with multi-class classification, ROC area-under-curve (AUC) proves to be difficult to interpret. This is because the model outputs

need to be converted to One vs. Rest outputs. It was decided that this was not useful in this multi-class case, as the other metrics described above were more than sufficient for evaluating the available models.

3.5 Experiment Procedure

3.5.1 Embedding Model

As discussed in section 3.3.2.1 the architectural hyperparameters were tested with fixed training hyperparameters to avoid polluting the results with improvements or regressions from non-architectural parameters. To determine the optimal training hyperparameters, a grid-search of five parameters was performed. Normally, an experiment must only have one independent variable to understand the specific effect of varying the value. However, for tuning a ML model this is not necessary, as the priority in this case is finding an optimal combination of hyperparameters as fast as possible. This means that we do not need to be able to understand the impact of each variable, and the experiment trades cross-variable independence for speed. The experimental variables for these experiments are shown below in Table 3.5-1.

Variable	Variable Type	Description
Loss	Dependent	Mean NEG loss over all validation set samples
Batch Size	Independent	Size of batches allocated per training step within an epoch
Learning Rate	Independent	Magnitude of gradient updates
Context Window	Independent	Number of positive context samples
Negative Multiplier	Independent	Number of negative context samples per positive context
Epochs	Control	Number of epochs to train model over dataset
Split Ratio	Control	Ratio of training to validation samples
Threads	Control	Concurrent threads per training process
Optimizer	Control	Algorithm used to update weights based on computed gradients

Table 3.5-1: Experimental Variables for Training Hyperparameter Sweep

To limit the total number of training cycles, each independent variable needs a limited range of values to test. These values are shown in table 3.5-2.

Variable	Possible Values
Batch Size	1024, 2048, 4096
Learning Rate	1.0e-2, 1.0e-3, 1.0e-4
Context Window	4, 5, 6
Negative Multiplier	2, 4, 5

Table 3.5-2: Possible Values of Independent Variables for Training Hyperparameter Sweep

Since there are three values per hyperparameter, this totals to $3^4 = 81$ possible combinations. The model runs are handled through Rust's integration test system. The test handles a single model run, which resembles the trainer binary without the added complexity of CLI parsing. The configuration combinations are generated through a configuration iterator which uses the *itertools* library to generate a cartesian grid with the four parameters. The specific instance is selected through an environment variable

called “CONFIG_ID” which ranges from 0-80 and can be used to fetch the n th configuration from the iterator with $O(n)$ time complexity, avoiding the need to compute the entire grid for each configuration (because Rust iterators are lazy). Since the test only handles single tests, a third-party job scheduler is needed to parallelize and handle the training processes. GNU Parallel (Tange, 2026) is a UNIX tool used in conjunction with the 'seq' command to iterate through configuration IDs and spawn test processes with cargo. The specific command is shown below.

```
$> seq 0 80 | parallel -j 8 CONFIG_ID={} cargo test --release --test sweep
```

This spawns 8 concurrent processes which use 8 threads for training each, less than the 128 available on the ‘Linebreaker’ server. However, 16 parallel processes proved to be slower, since the main bottleneck in training is data copying from RAM to GPU VRAM. The results of this experiment are shown in section 4.

3.5.2 Classification Models

To evaluate the success of each classification algorithm, the models were each trained on the training set and predictions were generated with both the training and test dataset. This is because it is important to see the difference in performance by contrasting metrics from the training and test sets. This can be used to determine whether a model is overfitting, underfitting, or generalizing correctly.

Variable	Variable Type	Description
Algorithm	Independent	Multi-class classification algorithm used
Preprocessing Pipeline	Independent	Feature selectors and transformers utilized
Precision	Dependent	Rate of true positives over all predicted positives
Recall	Dependent	Rate of true positives over all actual positives
Specificity	Dependent	Rate of true negatives over actual negatives
Accuracy	Dependent	Rate of correctly classified samples over all samples
F_1 score	Dependent	Accuracy equally valuing precision and recall
F_2 score	Dependent	Accuracy valuing recall more (minimizing false negatives)
Dataset	Control	NF-UNSW-NB15-v3 NetFlow dataset

Table 3.5-3: Experimental Variables for Classifier Evaluation

It may become clear that the NPV has been omitted. This is because the dataset used (and IDS datasets in general) is highly imbalanced. Since there are significantly more benign samples, the number of true negatives for this class is highly inflated, which causes the NPV to be misleading. Even a badly performing model will show a significant number of true negatives for all classes that are not benign.

The dataset pipeline was adjusted to select 50% of the best scoring features for both categorical (with Chi-Squared) and numerical (with ANOVA) for one experiment, and then select all features for another. This should show if the feature selection was useful for reducing dimensionality while retaining model performance, or if removing features lost too much information and affected performance negatively.

3.6 Statistical Analysis

3.6.1 Embedding Model

Since the main determinant of the embedding model's performance is the classification models, the only metric to compare different hyperparameter configurations is the mean NEG loss across all validation set samples. Thus, to select the best configuration, the researcher simply chose the model with the smallest loss. Nevertheless, the architectural hyperparameter results in section 4.2 were qualitatively analysed with PCA and t-SNE projections to ensure learned embeddings were properly structured and contained useful information for downstream classifiers.

3.6.2 Classification Models

It is important to keep in mind that certain metrics will be affected by the benign dataset imbalance. As mentioned in the Experimental Procedure section, NPV has been completely discounted as it will not provide any real measure of performance. However, accuracy and specificity will also be affected. A higher specificity indicates that there are few false positives in respect to true negatives. For IDS datasets, false positives are not the primary concern. Rather, false negatives are significantly more important to minimize. False negatives in the benign class indicate attacks that have been missed by the model, which is the worst-case scenario. To that end, recall is the most important metric to evaluate, as it measures the true positives in respect to all actual positives. Precision is still key, as it represents all true positives over all predicted positives. This means that precision in a multi-class model indicates how well each malicious class is predicted.

As a final note, when comparing the performances of these models with the papers mentioned in section 2, remember that most previous work deals with binary classifiers. Thus, accuracy is still relevant, as a true negative in the benign class for a multi-class model is the same as a true positive in the malicious class for a binary model. This means that ultimately the accuracy values of these multi-class classifiers are comparable to the

accuracy values of previous works; with the caveat of accuracy being useless for comparing the performance of individual classes.

4 Results

4.1 Preparation Work

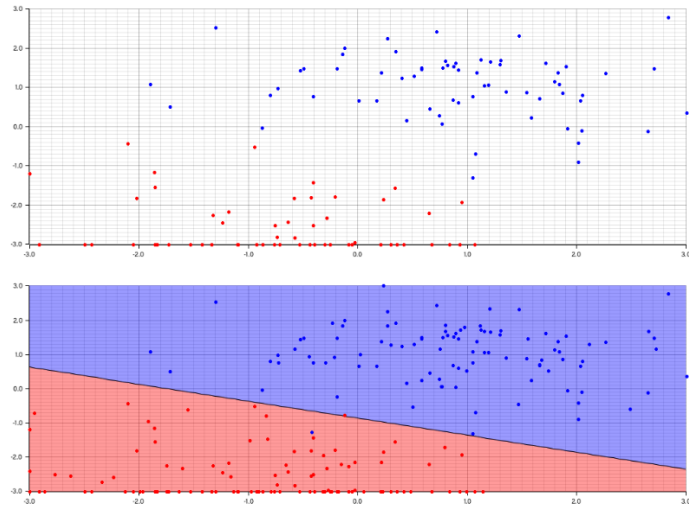


Figure 4.1-1: Linfa Logistic Regression Output

Figure 4.1-1 shows the output of the simple experiment explained in section 3.2, implemented with Linfa. This implementation required six imports from three different Linfa sub-libraries.

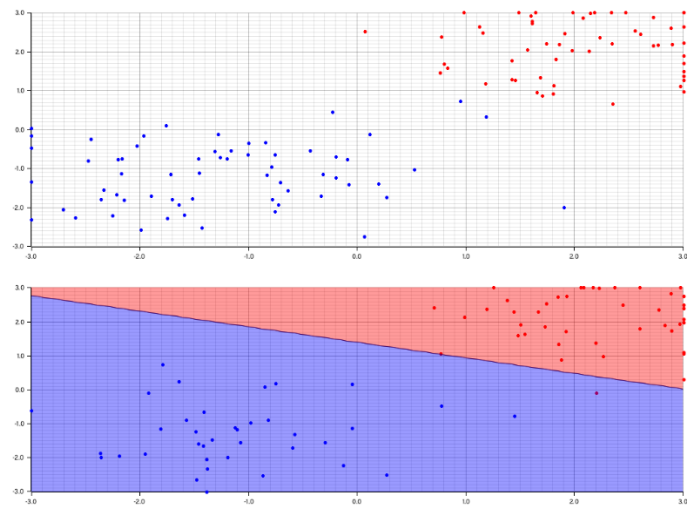


Figure 4.1-2: SmartCore Logistic Regression Output

Figure 4.1-2 shows the output of the same experiment implemented with SmartCore. There is no real difference in output, despite the randomly generated points being in

different positions causing the decision boundary to change. This implementation only required five imports from the monolithic SmartCore library.

The following table shows the aggregated performance in both memory usage and runtime for all three libraries.

Library	Peak Memory Usage (MB)	Mean Runtime (ms)
scikit-learn	47.32	974
Linfa	3.303	22.5
SmartCore	3.314	19.8

Table 4.1-1: Preliminary Experiment Performance Results

4.2 Architectural Hyperparameters

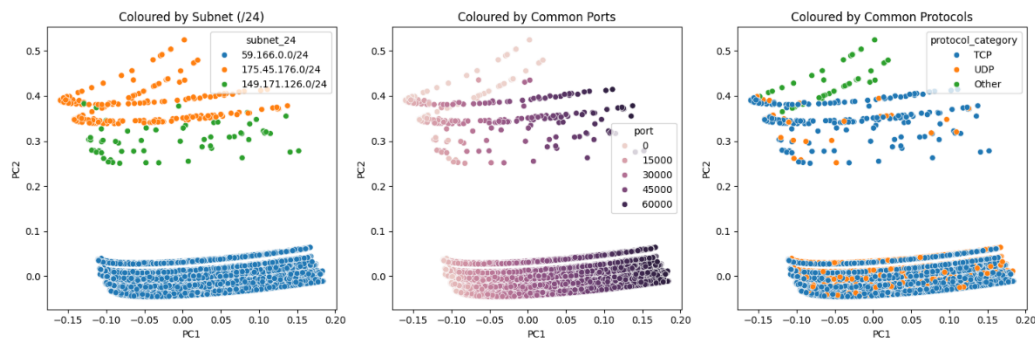


Figure 4.2-1: Initial PCA-2 Projections (All)

Figure 4.2-1 shows the PCA-2 projections of the embeddings (subsampled to 15,000 for clarity). There is some subnet structure along PC2, so it is safe to assume that PC2 roughly represents how the model embeds subnet context. Another key insight from these plots is that protocols labelled as “Other”, emerge as a curve moving away from the parallel lines. This can be confirmed by plotting the PC3 component.

PCA-3 Projection

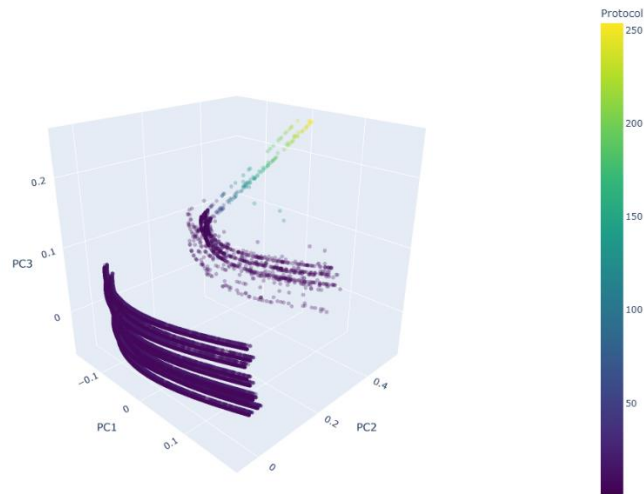


Figure 4.2-2: Initial PCA-3 Projection (Protocol)

It is important to note that this data was gathered with an earlier version of the IP2Vec implementation (specifically commit 2c639c6), so there are some samples that were pruned at the dataset validation stage, despite being generated from the inference binary. This issue is resolved from commit 7758736 onwards. To view the port and protocol structure more clearly, Figure 4.2-2 shows the same PCA-2 projections, zoomed into only the 59.166.0.0/24 subnet.

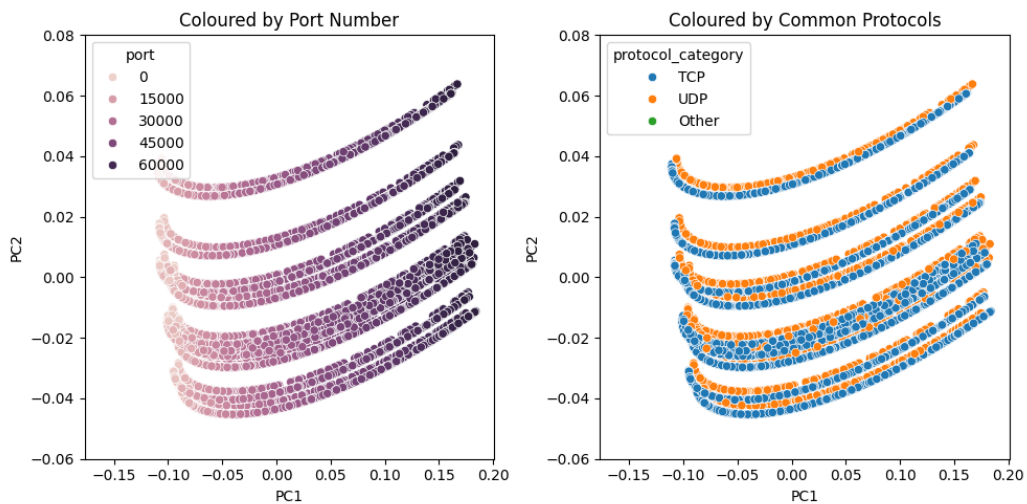


Figure 4.2-3: Initial PCA-2 Projection, Single Subnet (Port and Protocol)

Now, it is clear that the port numbers are arranging from smallest to largest along each line, and that TCP and UDP are treated as similarly related parallel lines. With the corrections detailed in Section 3.3.2.1, the following PCA-2 plots were generated.

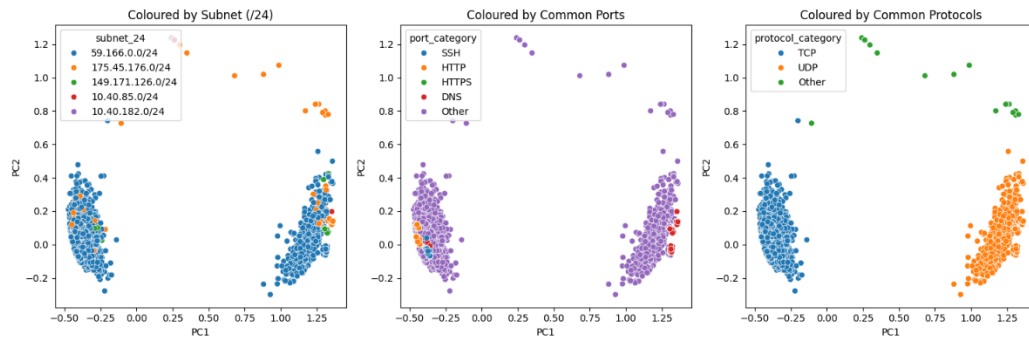


Figure 4.2-4: Corrected PCA-2 Projection (All)

Here, protocol is clearly clustered into the three main groups, and it is possible to see the ports are also forming sub-clusters within the protocol groups. Subnet variance is affected by these changes, and it is difficult to see the complex relationship with only three principal components. Therefore, a t-SNE projection was also generated from this data (with PCA-64 preprocessing).

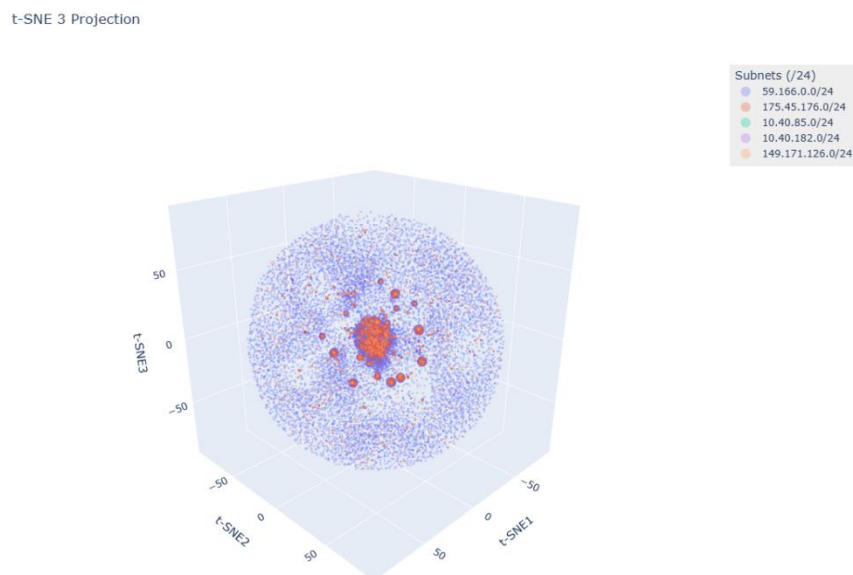


Figure 4.2-5: Corrected t-SNE-3 Projection, Coloured by Subnet (/24)

Figure 4.2-5 shows that subnets are still clustering in higher dimensions, as t-SNE derives the projection from the local structure of the points, rather than truncating lower variance dimensions.

4.3 Embedding Hyperparameter Sweep

To train all 81 configuration combinations, it took 21 hours. Appendix B shows a faceted heatmap of the grid search described in section 3.5.1. These plots show that smaller batches, larger learning rates, and larger context windows all yield lower loss values. The final tested parameter is different, increasing the negative multiplier has diminishing returns, especially at larger context windows. The best iteration was trial 6, with a loss of 2.90×10^{-3} . This is within 2×10^{-5} of the same configurations with higher negative multipliers. This configuration had the minimum batch size (1024) the maximum learning rate (1×10^{-2}), the maximum context window (6), and the minimum negative multiplier. The worst configuration proved to be trial 72, with batch size 4096, learning rate 1×10^{-4} , context window of 4, and negative multiplier of 2.

4.4 Classification Models

After comparing all four model outputs (Linear SVM and three GBDT variations) with 50% of numerical and 50% categorical features vs. all features, it was determined that in all models, pipelines which did not remove any features were significantly more performant. See Appendix C for full metric scores to four significant figures.

Note that the Overall class in the following figures is calculated as the average of all the class scores for a particular metric. Figure 4.4-1 shows the metric outputs of the linear SVM classifier.

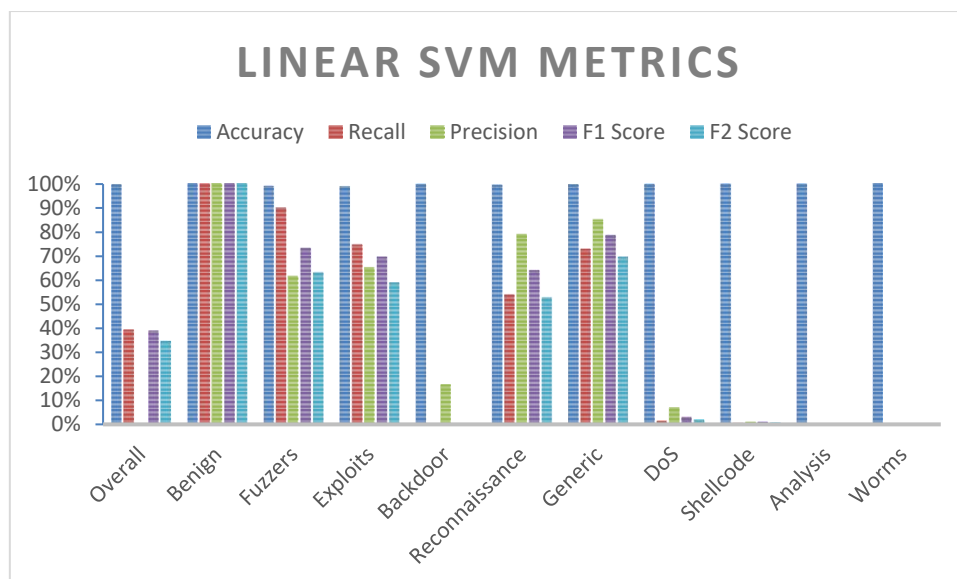


Figure 4.4-1: Linear SVM Metrics by Class

Here we can see that the accuracy of all classes is roughly 99%, which is expected, as was mentioned in section 3.6.1. As can be seen, the worst performing classes for this model are Backdoor, DoS, Shellcode, Analysis, and Worms. This model predicted two false positives in the Benign class; which proved to be one Exploit sample and one Shellcode sample.

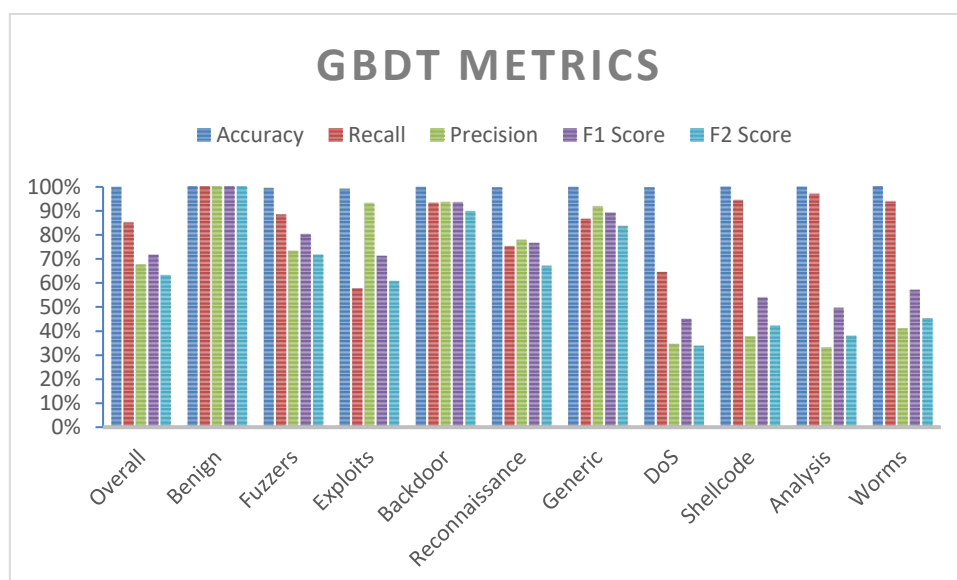


Figure 4.4-2: GBDT Metrics by Class

Figure 4.4-2 shows the standard multi-class GBDT model's performance. The overall performance of the model is better than the linear SVM. Furthermore, the GBDT model

seems to be significantly better at classifying the Backdoor class. This means that the worst performing classes for this model are DoS, Shellcode, Analysis, and Worms. Notice that the recall scores for these classes are still above 90%, this means that the classifier correctly predicts most actual positive values, but since precision is low, it means the classifier is prone to false positives. This model had zero false positives in the Benign class, meaning that all malicious samples were classified as some malicious class (even if it was incorrect).

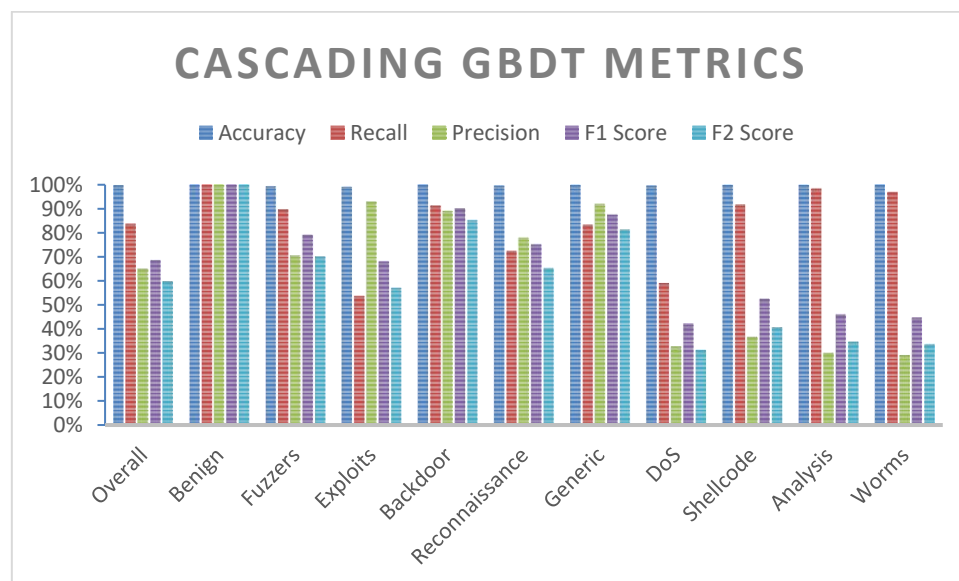


Figure 4.4-3: Multi Cascading GBDT Metrics by Class

Figure 4.4-3 shows the metrics for the cascading GBDT model with the multi-class fine grained classifier. The results are extremely similar to the standard GBDT; however, this model performs slightly worse in all metrics. This classifier had four false positives for the benign class, two Exploit samples and two Generic samples.

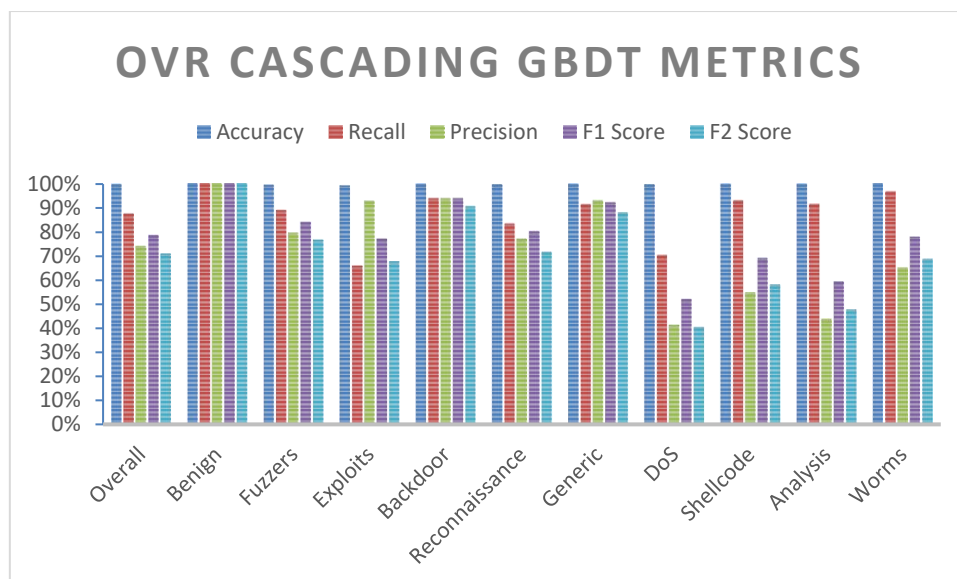


Figure 4.4-4: One vs. Rest Cascading GBDT Metrics by Class

Figure 4.4-4 shows the same cascading GBDT model utilizing 10 binary classifiers for One vs. Rest multi-class classification. Across the four worst performing classes, there is at least a 9% increase in precision from the standard cascading classifier. The most improved class is Worms, with a 36% increase in precision. In addition, there is marginal improvement across the board for all metrics. Finally, the model classified two malicious samples as benign, one Reconnaissance sample and one Shellcode sample.

5 Discussion

At this point, the report evaluates the performance of the models and interprets the results set out in the previous section. However, any interpretations of how a ML model learns and processes data are purely speculative. It is impossible to fully understand what a model is doing as they behave like a black box. Findings in this section are all backed up with data from the previous section, but it must be noted that there cannot be any concrete explanation of the models reasoning.

5.1 Architectural Hyperparameters

The initial model architecture shown in Figures 4.2-1 and 4.2-2 demonstrate some level of embedding learning, as each axis correlates with a specific feature (PC1 relates to ports, PC2 to subnet/hosts, and PC3 for protocols). However, these embeddings did not learn any relationships among ports and protocols, as they were simply ordered from smallest to largest. This is considered a degenerate embedding as it provides no additional information to downstream classifiers. This diagnosis was supported by the training behaviour of the model, which would start with an extremely high loss at the first batch (~190) and would cause an exploding gradient which would compress the embeddings into the anisotropic structure highlighted in section 4.2. The loss would then plateau at 1.38 from epoch 1 until the end of training, demonstrating no further gradient updates and no adjustments in the embeddings for most of training.

Once linear layers were replaced with embedding layers for the port and protocol inputs, the latent representations now reflect implicit reasonings such as certain protocols like HTTP and HTTPS being related or SSH and Telnet being similar. This is extremely helpful to the classifiers as they now have important information that can help with identifying malicious network traffic without any additional complexity in dataset preprocessing or model parameters.

Nonetheless, with these improvements, there are downsides. Overall variance across dimensions is significantly lower after these changes, which makes host/subnet

grouping difficult to see in PCA-2 and PCA-3. The t-SNE plot (Figure 4.2-5) allows the evaluator to view a representation of local structure without truncating lower variance dimensions. This shows clear subnet structure is preserved in higher-order dimensions, and no information has been lost. With these results, it can be stated that all information from the original dataset has been preserved, and complex non-linear intuitions were learned by the model.

5.2 Embedding Hyperparameter Sweep

For this experiment, training times (epochs) were fixed across all iterations. This can likely explain certain trends in the results. Firstly, larger batch sizes seem to correlate with higher loss values. This is possibly because larger batches mean fewer batches and less frequent gradient updates. Learning rates are similar since smaller learning rates make for smaller gradient updates. Naturally, this means that these models would need more time to converge to the loss minima.

It was necessary to pin the epoch parameter to reduce the amount of configuration combinations. By adding three possible values for the epoch parameter, the total combined training time would at least triple. This would likely be more than three times as the main limiting factor of parallel training is memory transfer from RAM to GPU, as discussed in section 3.5.1. Note that the best configuration has all parameters at the extremes of their value range. This is a clear sign that the absolute optimum configuration has not been found. The experiment would benefit from a larger sweep range or shifting of the window towards the better direction. This is discussed further in section 6.2. Another important thing to note is that larger batch sizes use less of the available performance capacity of the ‘Linebreaker’ GPUs. This was discovered in a smaller scale experiment with batches of size 64 which maxed out both GPUs at 8 parallel jobs, whereas the batch sizes of the actual experiment peaked at around 50% usage on both.

5.3 Classification Models

All models achieved an accuracy of over 99% on all categories. This was expected because, as described in section 3.6.2, the class imbalance of benign samples inflated the true negative scores on malicious classes. However, when comparing with previous work, this accuracy is comparable. Across all models, the average overall accuracy was ~99.7%, which is better than the best binary classifier of Fosić *et al.* (2023), which had an accuracy of 99.1%. The other overall metrics are not comparable with binary classifiers, as the imbalanced malicious classes skew the values lower. Gonzalez Granadillo and Kaaniche (2025) performed similarly, with their Random Forest model having the best accuracy of 99.8%. It is important to note that the *overall* accuracy was calculated by taking the average of all the classes' accuracy. This was done to contrast of multi-class classifiers with binary classifiers. The accuracy metric which is closest to the accuracy metrics of binary classifiers is actually the accuracy of the Benign class, which was never lower than 99.99% for any model. Regardless, this implementation's multi-class models all outperform Gonzalez Granadillo and Kaaniche's multi-class Random Forest (99.5% accurate).

Out of all the tested models, the One vs. Rest cascading GBDT is clearly the best model, and the linear SVM is the worst. Interestingly, the Backdoor class performed very poorly with the linear SVM, but all the GBDT models performed very well with the same class. This is likely because the Backdoor class has significant feature overlap with other classes, making it difficult to linearly separate, giving the non-linear GBDT models an advantage. Upon analysing the confusion matrix, the most misclassified classes for Backdoor samples were the Fuzzers and Exploits classes, followed by the Reconnaissance and Generic classes. This hypothesis is further reinforced by discussing the preprocessing pipelines. When feature reduction was attempted, GBDT models struggled with the same issues of Backdoor misclassification. This indicates that the removed features were highly correlated with Backdoor samples. This means that the feature selection pipeline was completely counterproductive. Nevertheless, the dataset transformers accomplished the goal of making the model's input data interpretable and

learnable. Another interesting note on the Backdoor class is that it has the second fewest samples of any class. This makes the GBDT performance with this class even more impressive. Finally, one might notice that the Exploits class has a low recall across all models, this is likely due to the label being slightly too broad and leaking into the other classes. The confusion matrices reveal that this class is most often misclassified as Fuzzers, Reconnaissance, and DoS. This is a weakness of the NF-UNSW-NB15-v3 dataset that was not recognized before implementation.

The linear SVM completely failed to correctly classify any Analysis and Worms samples. This can be explained, as these classes have the least samples of all classes (excluding Backdoor), with Analysis having 2,381 samples and Worms having just 158 samples. This heavy class imbalance explains the poor performance. DoS and Shellcode classes had 5,980 and 4,659 samples respectively. This makes them the second most underrepresented group of classes, compared to the rest of the classes, which had 5-digit sample sizes. Taking everything into account, the linear SVM model proves to be unfit for the purpose of multi-class anomaly detection. This is not to say the implementation was a complete failure, as the dense IP embeddings likely allowed for a larger margin of benign and anomalous samples in the feature space. This would explain why it outperformed the SVM implemented by Fosić *et al.*, (2023) and Gonzalez Granadillo and Kaaniche (2025) in terms of accuracy.

All the GBDT classifiers had an accuracy within 0.08% of each other, this is to be expected as they utilize the same underlying algorithm. The worst of these was the standard cascading GBDT model, although the difference is almost negligible. Furthermore, one should prefer the non-cascading GBDT, as it trains in a fraction of the time (since it is one model compared to 11) for similar performance. There is not much difference in the badly performing classes either. This is because the class imbalance issue is not unique to the Benign class, rather the malicious classes also suffer (as discussed above). The difference in false positives to the Benign class is inconsequential, as this minimal difference (zero vs. four out of 2.3M) is not statistically significant. The improvement in the OVR cascading GBDT is significant, especially for

precision scores (and therefore the F_β scores). Despite inverse class weighting being applied to all the GBDT models, this does not necessarily translate into the tree splits. This is because in a multi-class DT, a split that helps many classes (like common classes) is favoured to a split that helps one class. When the problem is decomposed into several binary classifiers, which means that each split will explicitly favour its assigned class. Regarding inverse class weighting, it is possible to notice that for badly performing classes such as shellcode, analysis, and worms, the recall scores are matching or even better than other classes. This is because inverse weighting will make the model prefer to predict underrepresented classes. This greatly helps with false negatives, which is what recall measures, but can cause other classes to be misclassified as these small classes. Which causes the precision on these classes to be comparatively low. Nonetheless, this does not affect the recall of other classes very strongly, as they have more samples with stronger signals which have proven to be correctly classified.

This project aimed to improve ML-based NetFlow anomaly detection with two major changes: an IP embedding model to enhance IP data to be usable with classification algorithms, and utilization of multi-class model outputs for greater actionability. First, the IP embedding model that was implemented succeeded in learning contextual network information and representing it in a dense latent space. This was proven through analysis of different projections generated with two dimensionality reduction techniques, showing a clear spatial representation of abstract concepts such as similar protocols, common port and protocol pairs, and subnets. However, the hyperparameter sweep for the embedding model failed to reach a clear optimum configuration, likely due to a sweep range that was too narrow or fixed epoch lengths.

Next, the classification models strongly outperformed the classifiers described in all previous work mentioned. However, the multi-class statistics were not as impressive. Specifically, the linear SVM model was completely inadequate in correctly classifying malicious samples. The GBDT models performed significantly better, due to their non-linear capabilities. Of these three variations of GBDT, the OVR cascading model was meaningfully better than the other two. Nevertheless, the F_β scores for malicious classes remained unsatisfactory, due to the severe class imbalance present in the chosen dataset. Therefore, this report concludes that the goal of making anomaly detection more actionable was not convincingly met.

Finally, two unexpected conclusions can be drawn from this research. First, the issues with *SmartCore* and *Linfa* proved that the Rust ML ecosystem is still too immature for a project of this scale. Second, the feature scoring element of the preprocessing pipeline was detrimental to the performance of the models. This is likely due to the statistical functions chosen for the feature scoring being unfit for the type of data present in NetFlow datasets.

6.1 Future Work

Further testing should be performed, to more comprehensively understand the effects and limitations of the embedding model. Specifically, it is important to determine whether embedding landmarks remain in similar positions when trained on different datasets. This would determine whether models trained with synthetic datasets (like the one used for this implementation) can be utilized in other environments, by simply retraining the embedding model with different network data. This would cement the efficacy of IP embeddings by allowing learned IP relationships to be transferable. This is something that was identified as a key driver in the omission of this type of data in previous work.

In addition, further classification algorithms can be tested utilizing the same anomaly detection framework, due to the nature of the generic trait definitions. If more algorithms are tested utilizing these techniques, it is possible a classifier that is resilient to extreme class imbalances could be found.

Another extension of this work could be implementing different feature scoring algorithms. As discussed in the previous section, the feature scoring algorithms failed to preserve performance when reducing the dimensionality of the feature space. More complex techniques such as wrapper scorers (which utilize predictive models to score feature subsets) or embedded methods (which integrate feature scoring as feedback loop to the actual model) could prove to be significantly more successful in reducing dimensionality.

Lastly, further work needs to be done to improve the performance of general-purpose ML libraries in the Rust ecosystem. Whilst Rust excels in low-level performance, memory safety, and expressive data structures, the optimization of general-purpose ML libraries in Rust leave much to be desired. Although bindings are available for many well-known C++ libraries, the dynamic linking and bindgen requirements make these obtuse and fragile to work with. Furthermore, libraries like *XGBoost* often do not maintain official

bindings, which can lead to competing standards with varying levels of popularity and usability.

List of References

A realistic cyber defense dataset (CSE-CIC-IDS2018) (no date). Available at: <https://registry.opendata.aws/cse-cic-ids2018/> (Accessed: 15 March 2026).

Bonaccorso, G. (2018) *Machine learning algorithms*. Second Edition. Birmingham Mumbai: Packt.

Casey, A. (2022) ‘Performance of serialization libraries in a high performance computing environment’. Available at: <https://hdl.handle.net/10657/13140> (Accessed: 25 March 2026).

Chen, T. and Guestrin, C. (2016) ‘XGBoost: A Scalable Tree Boosting System’, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. Available at: <https://doi.org/10.1145/2939672.2939785>.

CyberEdge Group (2025) ‘Main network security technologies currently in use or planned for acquisition by organizations worldwide in 2024’, *Statista. Statista Inc* [Preprint]. Available at: <https://www.statista.com/statistics/1319741/global-network-security-deployment/>.

DMLC (2025) *Notes on parameter tuning — xgboost 3.2.0 documentation*. Available at: https://xgboost.readthedocs.io/en/stable/tutorials/param_tuning.html (Accessed: 3 May 2026).

Edmonson, M. and Bromiley, M. (2024) *SANS 2024 AI Survey*. Available at: <https://www.hackerone.com/resources/pf/col/home/sans-2024-ai-survey> (Accessed: 6 May 2026).

Flanagan, K. et al. (2017) ‘Self-configuring NetFlow anomaly detection using cluster density analysis’, *2017 19th International Conference on Advanced Communication Technology (ICACT)*. *2017 19th International Conference on Advanced Communication*

Technology (ICACT), pp. 421–427. Available at: <https://doi.org/10.23919/ICACT.2017.7890124>.

Fosić, I. et al. (2023) ‘Anomaly detection in NetFlow network traffic using supervised machine learning algorithms’, *Journal of Industrial Information Integration*, 33, p. 100466. Available at: <https://doi.org/10.1016/j.jii.2023.100466>.

Gonzalez Granadillo, G. and Kaaniche, N. (2025) ‘AI-based anomaly detection and classification of traffic using NetFlow’, *SECRYPT 2025: the 22nd International Conference on Security and Cryptography*. Bilabo, Spain: SCITEPRESS, pp. 644–649. Available at: <https://doi.org/10.5220/0013552700003979>.

Guyon, I. and Elisseeff, A. (no date) ‘An Introduction to Variable and Feature Selection’.

Hendrycks, D. and Gimpel, K. (2023) ‘Gaussian Error Linear Units (GELUs)’. arXiv. Available at: <https://doi.org/10.48550/arXiv.1606.08415>.

Hotelling, H. (1933) *Analysis of a complex of statistical variables into principal components*. Available at: <https://psycnet.apa.org/record/1934-00645-001>.

Karagiannis, T. and Mortier, R. (2008) ‘Address and traffic dynamics in a large enterprise network’, *2008 16th IEEE Workshop on Local and Metropolitan Area Networks. Metropolitan Area Networks (LANMAN)*, Transylvania, Romania: IEEE, pp. 102–107. Available at: <https://doi.org/10.1109/LANMAN.2008.4675852>.

Kilian, S. et al. (2025) ‘CasinoLimit: an offensive dataset labeled with MITRE ATT&CK techniques’, *Proceedings of the 28th International Symposium on Research in Attacks, Intrusions and Defenses*. Gold Coast, Australia. Available at: <https://hal.science/hal-05224264> (Accessed: 15 March 2026).

Klabnik, S., Nichols, C. and Krycho, C. (2026) *The Rust programming language*. 3rd Edition. San Francisco, CA: No Starch Press.

Koda, S. et al. (2020) ‘Anomalous IP address detection on traffic logs using novel word embedding’, *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*. *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 1504–1509. Available at: <https://doi.org/10.1109/COMPSAC48688.2020.00-42>.

Lanvin, M. and Majorczyk, F. (2025) ‘Get out of DEDALE with RESCOUSSE: a new dataset and testbed for evaluating the detection of APT attacks among network and system logs’, *Computer Security. Esorics 2025 International Workshops: Anubis 2025, Secai 2025, Secassure 2025, Stmus 2025, Toulouse, France, September 22-24, 2025, (Lecture Notes in Computer Science #1623)*. Toulouse, France, pp. 1–21. Available at: <https://hal.science/hal-05329482> (Accessed: 15 March 2026).

Liu, L. et al. (2022) ‘Error prevalence in NIDS datasets: a case study on CIC-IDS-2017 and CSE-CIC-IDS-2018’, *2022 IEEE Conference on Communications and Network Security (CNS)*. *2022 IEEE Conference on Communications and Network Security (CNS)*, pp. 254–262. Available at: <https://doi.org/10.1109/CNS56114.2022.9947235>.

Luay, M. et al. (2025) ‘Temporal analysis of NetFlow datasets for network intrusion detection systems’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2503.04404>.

van der Maaten, L. and Hinton, G. (2008) ‘Visualizing Data using t-SNE’, *Journal of Machine Learning Research*, 9(86), pp. 2579–2605. Available at: <http://jmlr.org/papers/v9/vandermaaten08a.html>.

Mikolov, T. et al. (2013) ‘Distributed representations of words and phrases and their compositionality’, *Advances in Neural Information Processing Systems*. Curran Associates, Inc. Available at: <https://proceedings.neurips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html> (Accessed: 13 October 2025).

Moustafa, N. and Slay, J. (2015) 'UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)', *2015 Military Communications and Information Systems Conference (MilCIS)*. *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6. Available at: <https://doi.org/10.1109/MilCIS.2015.7348942>.

Myneni, S. *et al.* (2023) 'Unraveled — A semi-synthetic dataset for Advanced Persistent Threats', *Computer Networks*, 227, p. 109688. Available at: <https://doi.org/10.1016/j.comnet.2023.109688>.

Nowell, A. (2025) *Are we learning yet?* Available at: <https://www.arewelearningyet.com/> (Accessed: 13 October 2025).

Pearson, K. (1901) 'On lines and planes of closest fit to systems of points in space', *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* [Preprint]. Available at: <https://www.tandfonline.com/doi/abs/10.1080/14786440109462720> (Accessed: 23 March 2026).

Ring, M. *et al.* (2017) 'IP2Vec: learning similarities between IP addresses', *2017 IEEE International Conference on Data Mining Workshops (ICDMW)*. *2017 IEEE International Conference on Data Mining Workshops (ICDMW)*, pp. 657–666. Available at: <https://doi.org/10.1109/ICDMW.2017.93>.

Roponen, E. *et al.* (2021) 'A literature review of machine learning techniques for cybersecurity in data centers', *2021 62nd International Scientific Conference on Information Technology and Management Science of Riga Technical University (ITMS)*. *2021 62nd International Scientific Conference on Information Technology and Management Science of Riga Technical University (ITMS)*, pp. 1–6. Available at: <https://doi.org/10.1109/ITMS52826.2021.9615321>.

Sharafaldin, I., Habibi Lashkari, A. and Ghorbani, A.A. (2018) 'Toward generating a new intrusion detection dataset and intrusion traffic characterization', *Proceedings of the 4th*

International Conference on Information Systems Security and Privacy. 4th International Conference on Information Systems Security and Privacy, Funchal, Madeira, Portugal: SCITEPRESS - Science and Technology Publications, pp. 108–116. Available at: <https://doi.org/10.5220/0006639801080116>.

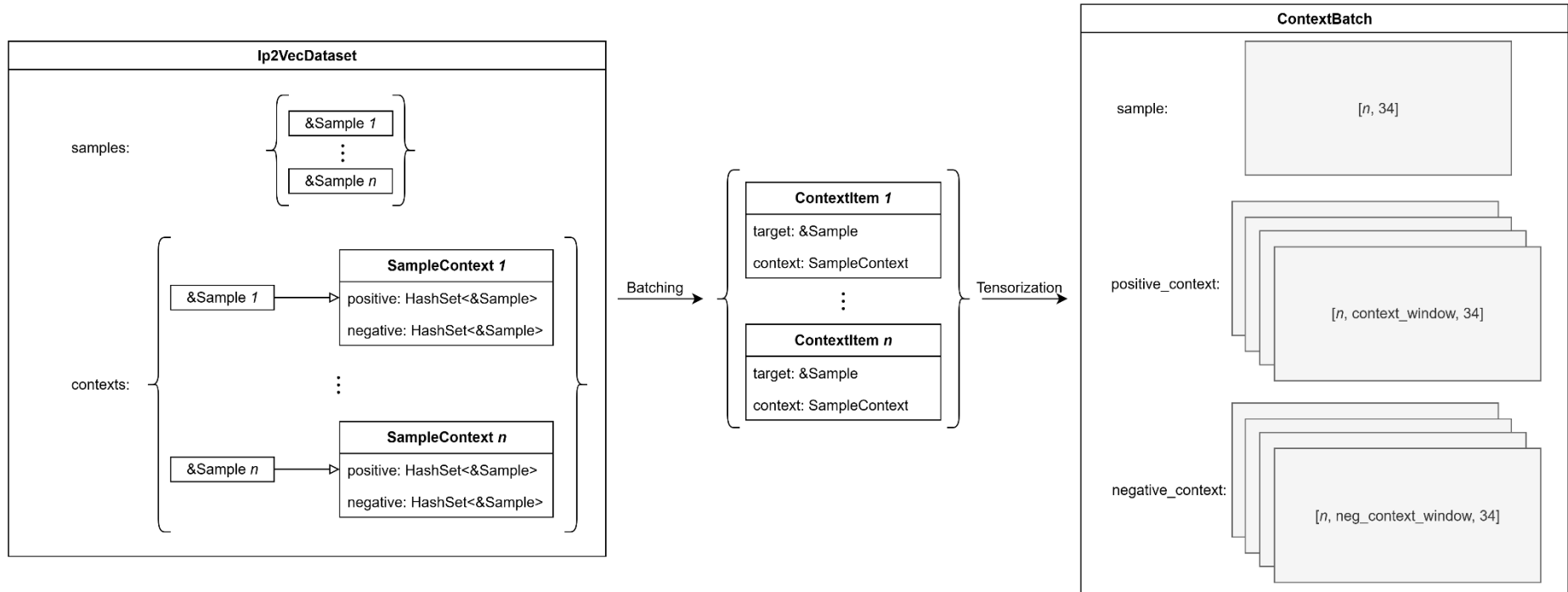
Statista (2026) *Enterprise Network Infrastructure - Worldwide*, Statista. Available at: <https://www.statista.com/outlook/tmo/data-center/network-infrastructure/enterprise-network-infrastructure/worldwide> (Accessed: 6 May 2026).

Tange, O. (2026) 'GNU Parallel 20260422 ('Artemis II')'. Zenodo. Available at: <https://doi.org/10.5281/zenodo.19701238>.

Vo, Q. *et al.* (2024) 'Detecting network anomalies in NetFlow traffic with machine learning algorithms', *2024 IEEE 49th Conference on Local Computer Networks (LCN). 2024 IEEE 49th Conference on Local Computer Networks (LCN)*, pp. 1–8. Available at: <https://doi.org/10.1109/LCN60385.2024.10639619>.

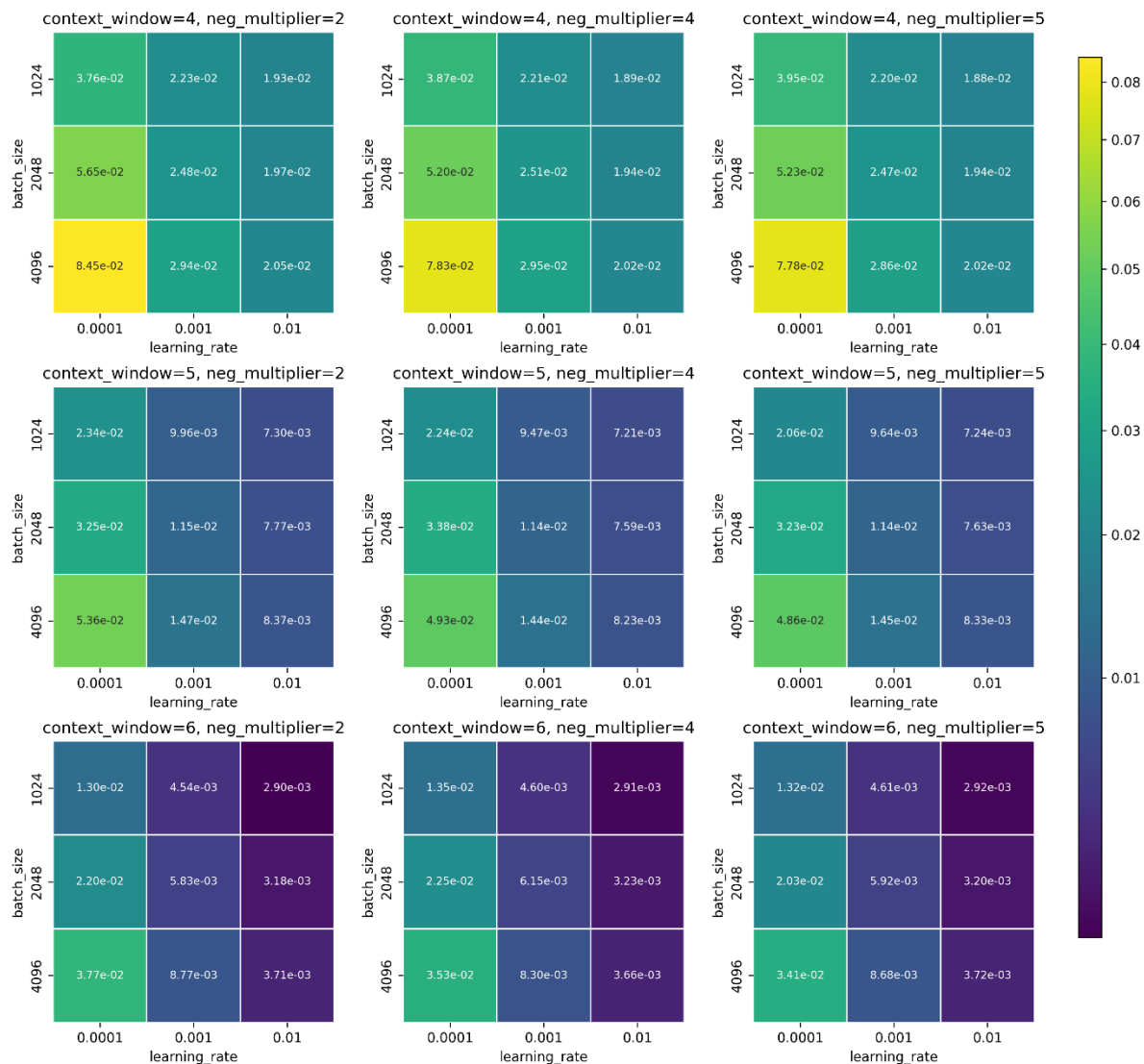
Appendices

Appendix A – Batching Data Flow Diagram



Appendix B – Embedding Hyperparameter Sweep

Faceted Heatmaps of Loss by Batch Size and Learning Rate



Appendix C – Full Metrics per Model

Class	Accuracy	Recall	Precision	F1 Score	F2 Score
Overall	99.64%	39.43%	-	39.06%	34.76%
Benign	99.99%	100.00%	99.99%	99.99%	99.99%
Fuzzers	99.05%	90.03%	61.72%	73.23%	63.10%
Exploits	98.80%	74.81%	65.32%	69.74%	59.03%
Backdoor	99.80%	0.21%	16.66%	0.43%	0.27%
Reconnaissance	99.56%	53.98%	78.99%	64.13%	52.78%
Generic	99.66%	73.03%	85.18%	78.64%	69.71%
DoS	99.75%	1.63%	7.03%	3.18%	2.01%
Shellcode	99.89%	0.65%	1.15%	1.24%	0.78%
Analysis	99.94%	0.00%	-	0.00%	0.00%
Worms	99.99%	0.00%	-	0.00%	0.00%

Table: Full Metrics of Linear SVM

Class	Accuracy	Recall	Precision	F1 Score	F2 Score
Overall	99.73%	85.09%	67.67%	71.65%	63.30%
Benign	99.99%	99.99%	99.99%	99.99%	99.99%
Fuzzers	99.37%	88.49%	73.39%	80.24%	71.73%
Exploits	99.13%	57.73%	93.04%	71.25%	60.77%
Backdoor	99.70%	93.27%	93.47%	93.37%	89.80%
Reconnaissance	99.66%	75.27%	77.95%	76.59%	67.15%
Generic	99.82%	86.61%	91.83%	89.15%	83.70%
DoS	99.60%	64.62%	34.75%	45.20%	34.01%
Shellcode	99.83%	94.29%	37.79%	53.96%	42.28%
Analysis	99.89%	96.92%	33.33%	49.60%	38.08%
Worms	99.99%	93.75%	41.09%	57.14%	45.45%

Table: Full Metrics of GBDT

Class	Accuracy	Recall	Precision	F1 Score	F2 Score
Overall	99.70%	83.67%	65.07%	68.49%	59.93%
Benign	99.99%	99.99%	99.99%	99.99%	99.99%
Fuzzers	99.30%	89.76%	70.57%	79.01%	70.18%
Exploits	99.07%	53.64%	92.92%	68.02%	57.07%
Backdoor	99.96%	91.33%	89.05%	90.17%	85.16%
Reconnaissance	99.64%	72.40%	77.82%	75.01%	65.24%
Generic	99.79%	83.33%	91.93%	87.42%	81.29%
DoS	99.57%	59.10%	32.71%	42.11%	31.25%
Shellcode	99.83%	91.73%	36.63%	52.35%	40.71%
Analysis	99.87%	98.41%	30.03%	46.02%	34.76%
Worms	99.98%	96.96%	29.09%	44.75%	33.61%

Table: Full Metrics of Multi Cascading GBDT

Class	Accuracy	Recall	Precision	F1 Score	F2 Score
Overall	99.78%	87.52%	74.22%	78.60%	71.02%
Benign	99.99%	99.99%	99.99%	99.99%	99.99%
Fuzzers	99.51%	89.04%	79.63%	84.07%	76.74%
Exploits	99.29%	65.90%	92.90%	77.11%	67.80%
Backdoor	99.97%	93.87%	93.97%	93.92%	90.61%
Reconnaissance	99.70%	83.46%	77.15%	80.18%	71.66%
Generic	99.87%	91.39%	92.95%	92.16%	88.02%
DoS	99.67%	70.29%	41.46%	52.16%	40.52%
Shellcode	99.91%	93.00%	54.96%	69.09%	58.28%
Analysis	99.93%	91.53%	43.99%	59.42%	47.79%
Worms	99.99%	96.77%	65.21%	77.92%	68.80%

Table: Full Metrics of OVR Cascading GBDT